Callum Lewis Hemsley

ch742@Sussex.ac.uk

278905

UNIVERSITY OF SUSSEX

Going Up – Unity 3D platformer

Final Project Report

8900 Words

01/05/2026

Abstract

This report aims to document the process of producing a 3D game within Unity. It will break down the planning of a project of this scale, development of ideas and plans, along with how features are chosen to be added into a system. Towards the end, it will evaluate the outcome of the project over its almost 8-month development time, and what would have been done to further the project given more time or resources.

Statement of Originality

This report is submitted as part requirement for the degree of "Computing for Digital Media and Games" at the University of Sussex. It is the product of my own labour except where indicated in the text. The report may be freely copied and distributed provided the source is acknowledged. I hereby give permission for a copy of this report to be loaned out to students in future years.

Table of Contents

1. Introduction

1.1. Project Overview

"Going Up" is a 3D "rogue-lite [1] platformer" with many inspirations throughout gaming. It began as a comedic take on the corporate ladder (i.e., "Going Up" the ladder), with a gameplay loop developed off games like Pizza Tower [2] and Lethal Company [3]. This project serves as a proof of concept for a far larger project, developing the core idea and identity of the project.

In 1980, a game was released called Rogue [4]. This game was very influential and eventually spawned a game genre known as "Roguelikes". These games use permanent death, turn-based combat and an infinite set of procedurally generated levels that the player must play through to ultimately gain a high score.

Eventually however, many games dropped the turn-based combat, and spawned a sub-genre known as "Rogue-lites", focusing on the infinite depth and procedural levels to create many different takes on the genre. These games will usually tap into other genres such as 4-player co-op multiplayers such as Lethal Company [3] or R.E.P.O, first person shooters such as Mycopunk and Buckshot Roulette, or even card games like Balatro.

For Going Up, I am aiming to make a 3D platformer with rogue-lite elements (i.e. procedural generation [5] and perma-death) where the player must break as many items and props as possible and make it back to the beginning of the level before the timer runs out.

1.2. Project Aims

For this project, there are three main aims that I intend to explore in the gameplay: unique movement systems that engage the player, procedurally generated levels to make gameplay endless, and customizable gameplay styles to expand accessibility.

These key focuses aim to develop an endless set of procedurally generated levels [6] that complement the movement systems of the game. Each level/floor should have very different layouts, but the player must be able to develop routes and plans to make their way through each level. I also aim to challenge the Rogue-lite genre in a way that not many other games have so far. Since Rogue-Lites rely on a random number generator (RNG) [7] to ensure that each level is unique, I am going to challenge the RNG system to make a unique challenge for the player and provide opportunities for the player to learn routes for themselves.

2. Professional & Ethical Considerations

2.1. Software testing

I am planning on testing in three main phases: System testing and then two phases of user testing. For the system testing, I am personally going to test code that I make by playing the game to test specific features, or the whole game to attempt to pick up on general bugs.

For User testing, the first build that I feel is presentable will be used for the first wave of user testing. Once this testing has been concluded, I will act on some of the feedback provided from the user testing and then run a second smaller user testing to see how the changes made have affected their opinions.

2.2. BCS Code of Conduct

Shown below are the BCs Code of Conduct [8], and a description of how I am planning to abide by each individual rule.

---

1. Public Interest - You shall:

> *a. have due regard for public health, privacy, security and wellbeing of others and the environment.*
> *b. have due regard for the legitimate rights of Third Parties.*
> *c. conduct your professional activities without discrimination on the grounds of sex, sexual orientation, marital status, nationality, colour, race, ethnic origin, religion, age or disability, or of any other condition or requirement.*
> *d. promote equal access to the benefits of IT and seek to promote the inclusion of all sectors in society wherever opportunities arise.*

For accordance with 1.a, a script (appendix section 6.4.2) will be read out that will properly inform the participant on what data is collected (i.e. any accessibility issues they struggled with, what questions they ask, and how they play the game) and will request verbal consent to move forward. The participant will also be given contact details to myself and my supervisor.

Additionally, any data recorded will be anonymised, with no name or other personal details attached. The data itself will also be stored securely and only accessed by myself.

For accordance with 1.b through 1.c, I plan on being entirely un-discriminatory and attempt to give equal access to the game regardless of any personal conditions. This will include those listed in 1.c.

---

2. Professional Competence and Integrity - You shall:

a. only undertake to do work or provide a service that is within your professional competence.
b. NOT claim any level of competence that you do not possess.
c. develop your professional knowledge, skills and competence on a continuing basis, maintaining awareness of technological developments, procedures, and standards that are relevant to your field.
d. ensure that you have the knowledge and understanding of Legislation and that you comply with such Legislation, in carrying out your professional responsibilities.*
e. respect and value alternative viewpoints and, seek, accept and offer honest criticisms of work.
f. avoid injuring others, their property, reputation, or employment by false or malicious or negligent action or inaction.
g. reject and will not make any offer of bribery or unethical inducement.

In accordance with point 2.a through 2.c, all work done will be within my professional competence, and any skill that I am unaware of/unable to do will be expressed and not undertook.

In accordance with point 2.d, all legislation will be followed throughout the project's lifetime.

In accordance with 2.e, any feedback given will be considered honestly and respected.

In accordance with 2.f, I will ensure testing is completed with a demo/product that has minimum reached most accessibility requirements. This includes issues to do with visual and auditory accessibility.

In accordance with 2.g, no bribery will be used to gain or alter participants responses, nor will it be encouraged.

---

3. Duty to Relevant Authority - You shall:

a. carry out your professional responsibilities with due care and diligence in accordance with the Relevant Authority's requirements whilst exercising your professional judgement at all times.
b. seek to avoid any situation that may give rise to a conflict of interest between you and your Relevant Authority.
c. accept professional responsibility for your work and for the work of colleagues who are defined in a given context as working under your supervision.
d. NOT disclose or authorise to be disclosed, or use for personal gain, or to benefit a third party, confidential information except with the permission of your Relevant Trustee Board Regulations Schedule 3 v8 Code of Conduct for BCS Members Page 3 of 5 Reviewed by Trustee Board 8 June 2022 Authority, or as required by Legislation.
e. NOT misrepresent or withhold information on the performance of products, systems or services (unless lawfully bound by a duty of confidentiality not

to disclose such information), or take advantage of the lack of relevant knowledge or inexperience of others.

(in this case, the "Relevant authority" mentioned is the University of Sussex).

In accordance with point 3.a and 3.b, I shall aim to meet the requirements of the University of Sussex by producing all deliverables at the designated deadline. Additionally, this will be done with respect to all parties and aim to always keep communication respectful.

In accordance with 3.c, I accept all responsibility for the project itself, alongside the outcome of the project.

In accordance with 3.d, all data personal to participants is completely stricken from any reports. Names, addresses or personal data will not be recorded; only their responses to the software are recorded. None of this data will be un-anonymised at any time.

In accordance with 3.e, the project will not be misrepresented to any parties, including the progress of the project, or the features implemented.

---

3. Duty to Relevant Authority - You shall:

a. carry out your professional responsibilities with due care and diligence in accordance with the Relevant Authority's requirements whilst exercising your professional judgement at all times.

b. seek to avoid any situation that may give rise to a conflict of interest between you and your Relevant Authority.

c. accept professional responsibility for your work and for the work of colleagues who are defined in a given context as working under your supervision.

d. NOT disclose or authorise to be disclosed, or use for personal gain, or to benefit a third party, confidential information except with the permission of your Relevant Trustee Board Regulations Schedule 3 v8 Code of Conduct for BCS Members Page 3 of 5 Reviewed by Trustee Board 8 June 2022 Authority, or as required by Legislation.

e. NOT misrepresent or withhold information on the performance of products, systems or services (unless lawfully bound by a duty of confidentiality not to disclose such information), or take advantage of the lack of relevant knowledge or inexperience of others

In accordance with point 4.a through point 4.c, I will professionally and respectfully complete the project and continue to align with the BCS code of conduct.

In accordance with point 4.d and 4.e, I will treat all members of the BCS in a respectful and professional manner.

3. Research and Requirements

3.1. Initial Research

3.1.1. Game - Pizza Tower

Pizza Tower [2] is an indie game released on the 26th of January 2023 developed by "Tour De Pizza". It features fast-paced platforming and uses a 1990's cartoon aesthetic, along with an energetic soundtrack. Each individual Level and World has unique visual styles and gameplay features (or gimmicks) with an accessible difficulty curve. The two features I will focus on in my analysis, however, are the movement system and the level layouts.

Each level consists of two main states – the first run through the level, and then an escape sequence. The player must navigate the level, complete puzzles or defeat enemies, until they reach the end. Then, they must then loop back through the level back to the beginning to escape. During the escape, alternative routes will open, and new enemies will spawn. The player must reach the entrance of the level at a certain time before a far more difficult enemy will spawn that can kill the player, making them restart the level.

The movement system takes a lot of inspiration from old Wario Land games [9]. Once the player begins a "sprint" by holding the shift key, the direction is locked. To switch directions, an animation plays where the player character completes something like a drift. Another key feature of the movement is the "Mach" system, where keeping a consistent direction and momentum, the players speed will surpass certain thresholds where the player can run through things; the higher the players Mach, the player can break more items or even avoid taking damage from certain sources.

Walk	Mach 2 (start)	Mach 3	Mach 4
			
Not in sprint yet, can't break through any objects. Turning is simple and instant.	Mainly used as a transition into a sprint, no additional abilities. Cannot turn from left to right, however.	Is now sprinting, can break through most smaller things and can defeat enemies. Turning is now done via drift mechanic.	Much faster now, can break though more things and damage from some enemies is ignored. Turning is still a drift.

Figure 1: Sprites from pizza tower - copied from "Spriters" [10]

One of the main features I wish to keep in mind as I develop Going Up is the Mach-based movement system. Having each speed milestone creates subsequent changes to how the player interacts with enemies and objects around them is interesting, and something I want to experiment with.

The locked direction when sprinting is also interesting, as the drifting the player must complete to change directions is something that could translate in an interesting way to 3D. Since the player will have 2 axis of movement instead of just the one while sprinting, the drift may feel like that of drifting a car – slowing down while taking large corners.

The biggest challenge however would be the 3D environment I'm planning on using. Since this movement system was built for 2D, the transition may not work as well as I expected, and more work may need to go into this.

3.1.2. Game - Mirrors Edge

Mirrors Edge [11] stands out to me mainly due to the visuals, and the way it approaches platforming. Mirror's Edge is a 3D parkour/adventure game made by DICE, released on the 13th of January 2009. For the first chapter of the game, and interspersed through many of the other chapters, the player must parkour over skyscraper roofs in a sprawling "utopian" megacity, sometimes while avoiding enemies.

Figure 2: Image from Mirror's Edge steam page [11]

The part that really stands out to me in the game is the very clear avenues that can be taken through the levels. In the image above, the viewer can clearly make out objects in the environment that can be used to get from the building in the bottom left to the top of the centre building [12]

For example, the flat board on the left just before the gap helps signal to the player that a jump can be made. The gap between the two air conditioners or the vent climbing the ledge of the centre building also adds a leading line that guides the players attention through and up the wall. This route looks something like this:

Figure 3: Annotated Figure 2

Using the environment to aid the player through the level by using props is far better than just using arrows or signs due to two main reasons: it's far more unobtrusive, and it keeps the player be more immersed. If the level design had arrows pointing the player where to go, they would become less immersed in the world since the player will be left wondering why the

arrows are on top of a skyscraper in the first place. This way, the player may not even notice they're being lead and be more inclined to congratulate themselves.

This then means that while playing the game, the player does not have to place too much focus on where they're meant to go next. They can instead commit more to reactions and the movement itself. Using this in Going Up, having clear directions that the player can move in means that the player can naturally know how to navigate forward at high speeds.

However, since my game will have back-tracking, the player might need to rely on their own method of finding their way back rather than relying on the map to show them the way.

3.1.3. Game - Lethal Company

Lethal Company [3] is a horror Co-op Roguelite made by "Zeekers", released on 23rd October 2023. In Lethal company, a group of 1 to 4 players must meet a money quota every 3 days by going into abandoned labyrinths and picking up scrap that they can sell. If they don't meet quota, the run ends, but apart from that the game theoretically cannot end if the players keep making more money to meet the growing quotas. However, beyond around quota 8, the game becomes near too impossible, as the quota is so high and there are far too many enemies that it becomes unfeasible to complete it.

Figure 4, 5: Images from Lethal Company's steam page [3]

Lethal company stands out to me because of the successful procedural generation system. On each planet, there's two main sections, the exterior and interior of the planets. The exterior is always the same every time you visit, acting as a transition from your ship into the structure. The interior is entirely random, generated every time you visit the planet.

The use of both preset environments and generated interiors has led to an incredibly successful gameplay loop; the player can learn the best methods to get from the landing location to the door but then must memorize the new interior layouts. If the whole map was generated however, there's far less reassurance that the player will be able to make any form

of progress. It's a two-part system of learning certain reliable paths and then using your skills to survive beyond that in unfamiliar areas.

Another feature as part of lethal company is the implementation of a weekly challenge map. The player goes into a level with a preset seed of the week with a leaderboard for how well each player has done. This reinforces learning pathing through the map, challenging the player to improve their skills in finding their way through the procedurally generated dungeons.

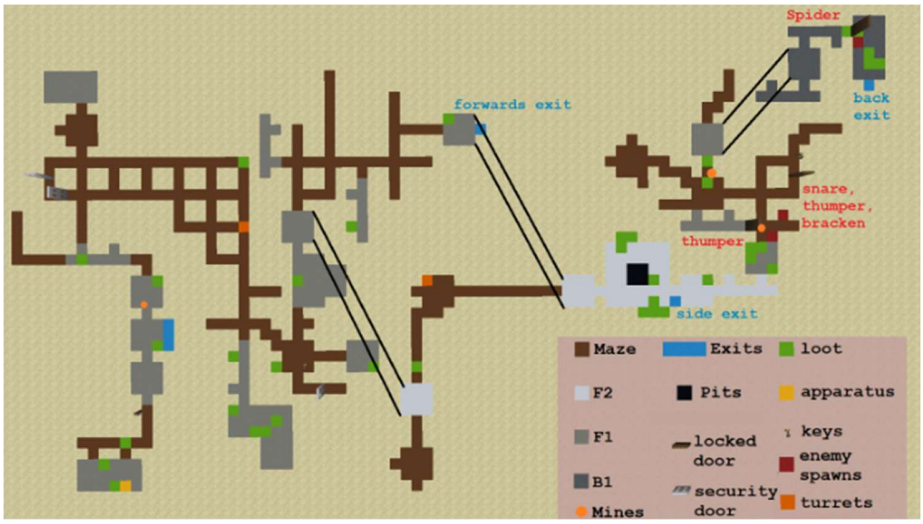


Figure 6: Example layout from Week 4's challenge "Muma-87". [13]

One of the more interesting systems to me is the link between the knowable and learnable versus the new and difficult areas the player must traverse. For example, the surface of each planet has a set layout that the player can memorize. For the weekly challenge, the game does this by using the same seed for their generation system, resulting in the same layout each time, allowing the player to learn it.

Applying this to Going Up, having some way of learning the floors, while keeping the randomness of them is something I will experiment with. As easy as it would be keeping each floor seed-generated (so each floor always has the same layout which is learnable) the player may become bored of the same layouts, and you miss out on a lot of randomness since the player must beat their previous score first. In conclusion, I need to balance between set seeds and the ability for the player to access new floors.

3.1.4. System – Quake Movement

One of the most influential games since they were invented is Quake [14], a game released by ID software [15] in 1996. It follows a collection of games such as Wolfenstein and Doom, two of the first ever FPS games (first person shooter), both of which introduced 3D environments to

many new players. Quake however built on the success of Doom massively, implementing better camera control, better optimization, but one thing stood out for many players at the time – the movement system. [16]

The Quake movement engine has gone down in history as one of the most influential systems in gaming history. It was one of the first games to implement jumping and physics to a player controller. Of course, 3D platformers existed long before this (such as Mario 64), but the strength for many other developers of the time was that the Quake game engine was both open sources, and very clean, meaning replicating this was very easy for developers.

For example, games that are known today for great movement systems are Team Fortress 2 and Titan Fall 2. Both games use a game engine known as the "Source Engine" [17] which uses a modified version of the Quake movement system.

The Quake movement system works by simulating a velocity for the player and using the dot product [18] of the players input and the velocity of the player controller to calculate the amount of movement that the player should experience. Key effects of this system are that the player takes a few frames to slow to a stop or speed up, so all movement feels smooth and controlled. Since it's all based on velocity too, external velocity impulses (such as an explosion or strong wind) can be added to have some built-in physics.

I would be implementing this into the Unity engine, so the collision calculations are done for me via the Character Controller component, however I have full control of how the player's input controls the velocity of the player. Doing this will make developing my custom movement system far easier, as the system can apply to many very different scenarios.

3.2. Requirements

3.2.1. Gameplay loop

The gameplay loop is built up of 4 main states: "Pre-game", "Elevator", "Run" and "Game Over". Once the player opens the game, they will enter the "Pre-game" and be given options to start a new game and start back at floor 1 building 1 or continue from their last save. Once either of these options are selected, they will enter the "Elevator" state. Here, they will have options to return to the pre-game (quit) or start the next level. They will also be able to see their game stats such as high score, current floor and building, their total score, etc.

Once the player enters the "Run" game state, the procedural generation system will be run, and the floors layout will be created. Once this is completed, the player will be able to access the movement system to control the player character. Their goal now is to break props around the office to gain points.

The player will have a set amount of time to collect a certain number of points to complete the level and return to the start of the level to make it into the elevator. If the player does not earn enough points, or doesn't reach the elevator in time, the player will lose and be sent to the "game over" state where their progress is lost. Alternatively, if the player does earn enough points, they will move to a new floor. The more points they achieve in the floor, the more floors they will move up.

Every so often, the player will reach a milestone and be sent to the next building. This has no purpose other than to act as a checkpoint for the player. For example, in building 1, they must reach floor 40 and then be sent to building 2, floor 1. Each building will also have a higher milestone.

3.2.2. Procedural Generation

The procedural generation system [6][7] is going to be used to create each individual office floor for the levels. It should generate a web of halls and offices that the player can move through, with destructible props to provide the points that the player needs to move on to the next level.

To do this, the built-in Unity RNG system will be used to make choices when needed. The Unity RNG system uses a set seed that is used to decide what is output next. If the same seed is used, you will always get the same outputs in the same order. Using this knowledge, I will use the current level's number (i.e. floor 1 building 1 or floor 36 building 3) as a seed, so that every floor will have a consistent layout each time the player visits it.

Overall, this system should produce office layouts that will stay the same for each floor. The outcome of this should mean the player can learn each individual floor, and plan which ones they want to visit, and which ones they want to skip due to personal preferences. This allows each player to have a set path through the game.

3.2.3. Unique player movement system

The movement system is going to be inspired by Pizza Tower's [2] movement system. The player will be able to walk in any direction, and pressing the shift key will enter the player into a "sprint mode". In the sprint mode, the player won't be able to turn as easily, but the players' high speed is drastically increased. If the player hits a wall while sprinting, they will be temporarily knocked back and stunned for a few seconds. To do this, I am planning on recreating Quake movement [16][19] as close as possible.

One of the ways that I'm planning on increasing the customization of the gameplay is to add different "movement styles". These styles will give the player different stats such as one overall balanced set, one with higher top speed and acceleration but lower control, less acceleration, etc. Ideally, each of these will come with their own set of animations, such as giving the player a skateboard or rollerblades.

To further the connections to Pizza Tower, the destructible props will be broken when the player runs into them with enough speed; different props will require different amount of speed. These props will provide the player with points to pass each individual floor. To complete this, the player will move through different phases of speed, each with their own respective control, max speeds and acceleration.

The sprint should also have a dash function, where the player can press the spacebar and increase the speed higher, but cannot turn at all during this – trading control for speed. Along with this to punish bad control, if the player ever has a head-on collision with a wall, the player should be stunned and stumble back.

3.2.4. Enemies and Upgrades

If these systems are completed with enough time to spare, I will start adding either enemies [20] or an upgrade system.

Enemies will spawn around the map like props, but will all have very different designs. One example would be a man on a wheely chair that will sit at their desk. If the player hits them, they will begin bouncing off props and walls at high speed, until they stop or disappear. If the player is hit by the chair while it's moving, the player will be hit backwards and stumble, like when the player hits a wall when sprinting.

An upgraded system will act as a way for the players to increase their passive stats, such as max speed, acceleration or movement control. To do this, the player must find a rare prop in the map somewhere and destroy it to earn an upgrade point. Once they find this, they can upgrade one of their stats by a small amount.

4. System Design & planning

4.1. Workflow

For the duration of the development, I am planning on using an agile approach to development, splitting key features into separate sprints. This means that I won't leave the last feature incomplete, forcing me to revisit code that I may not fully understand anymore.

I am planning on using Unity [21] version 6000.0.51f1, as it's the same version that is currently installed at the university labs, meaning I can work both from home, laptop, and at the university. To aid development on multiple devices, I'm going to use GitHub [22] to be able to commit my changes to the cloud. This also provides me with very accessible version control, so that I can see all the changes made in the past.

For 3D assets, I am going to use Blender [23] and Unity ProBuilder [24], while most assets will be downloaded from Sketchfab. I am going to mainly use Blender to edit existing assets, such as remaking UV unwraps or adding new animations, or export models as a "glb" file (the format used by Unity). ProBuilder is a plugin for unity that I can use to easily make 3D assets within Unity, saving a lot of time. I will use Photopea [25] to edit or curate new 2D assets and sound effects will be edited using Audacity [26].

4.2. Asset collection

The primary focus of this project is not the visuals, but instead the features such as movement and procedural generation. This means that not a lot of time will be spent creating new assets. Instead, I am planning on downloading models from sketchfab and then editing them to fit a consistent theme.

One key asset I am relying on for theming however is the music of the game. During the development, I had reached out to a producer "Golemm" [27] that I felt made music that would match the high-speed gameplay well, and they graciously allowed me to use their music within this game.

4.3. Architecture plans

4.3.1. Game States Controls

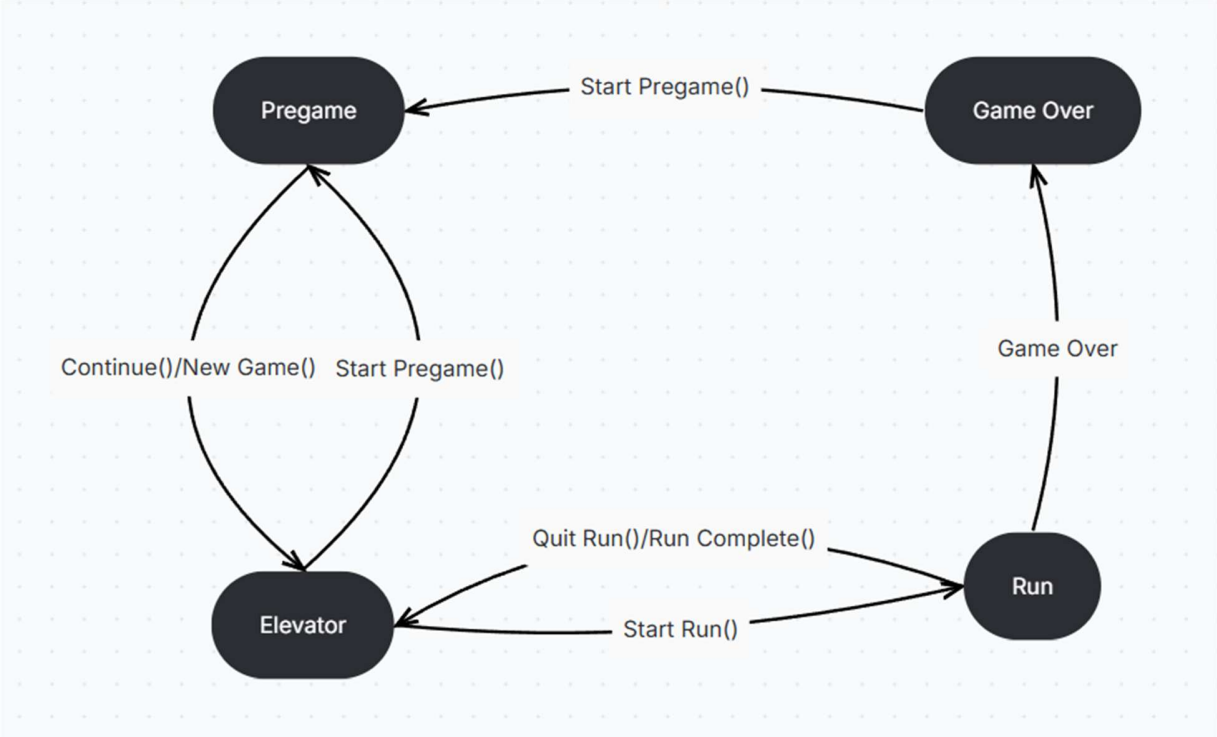


Figure 7: Game States Graph

Above is a graph showing all the transitions that the game will need to handle. Each of the states represent one of the game states. Different components will be able to listen to an event that will be called every time the state changes. For example, different sections of UI will enable/disable when a certain Game state is reached, or the player controller will reset once the state is no longer "Run".

On the Pregame Menu, three main options will be "Continue", "New game" or "Quit". If "Continue" or "New Game" button is pressed, the player will be taken to the Elevator. The save file will be loaded if the player continues but reset to default/start if new game is pressed. The Quit button will close the game application.

In the elevator's UI, the player will be able to either go back to the landing UI (Pregame) or start the next run. During the Run, the player will have the option to back out to the elevator via the pause menu, however this will lose their progress on that floor. If the player completes the floor, they will be returned to the elevator and keep their progress. However, if the player loses on that floor, the player will be sent to the game over screen, and their save file will be wiped.

4.3.1. Procedural Generation

For each floor of the game, I am going to use the floor number and building number as a seed for the RNG [7]. For example, Floor 1 Building 1's seed will be 00010001, and floor 50 building 25 would be 00500025. This means that if everything uses the same RNG, the floors will always have the exact same pattern.

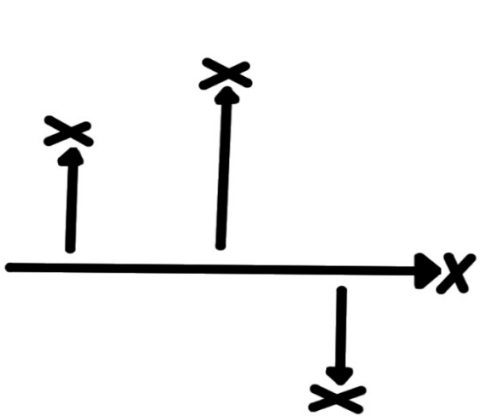


Figure 8: Representation of procedural generation branches.

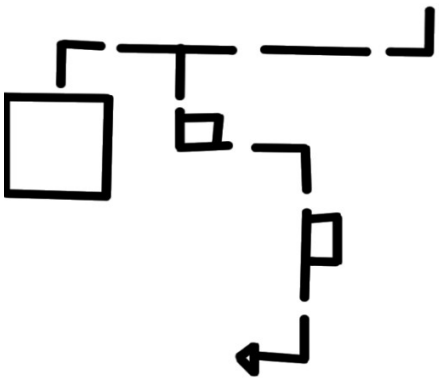


Figure 9: Representation of generation from above, including offices.

Above is a representation of the final generation of each individual floor's layout. Of course, the final version will have turns, but the focus of this is to show the "main branch" versus the "sub-branches". The main branch starts from the elevator and will create one long hallway. The subbranches will generate every so often, creating smaller halls along it. The X's represent large offices that will generate with a high number of props for the player to break.

To do this, the map will be generated on a grid recursively. Each section of hall will attempt to generate the next section until it either reaches the end of its branch, or the space it's attempting to generate is currently occupied. If it finds a valid direction, a new hall section is generated, otherwise, it will have to backtrack back down the hall until it finds a new valid direction. Each section can also meet certain criteria to attempt to make a second branch, so that once it's backtracked to, it will re-try the generation sequence. If the program ever backtracks all the way back to the start of the generation, the map has been completed.

The second diagram represents how each section is laid out on the grid. Each individual section will create connections between it and its neighbors that it is connected to. There are also representatives of the small and larger Offices that can generate. Towards the left of the image a large box shows the Large Office; it connects to the end of each branch and takes up the full space of a tile. Alongside some of the sections, a small Office is linked directly to the

Hall. These are smaller rooms that will contain a small number of Props. The Halls will also contain some Props along its edge.

4.3.1. Movement System

The key to the Movement System is the Movement Handler Component. This component will be used to store and access all the components needed, such as the input handler, the character controller, and all the accessible states, along with some helpful functions. Primarily, the Movement Handler directly controls the Unity Character Controller, setting its velocity on each frame.

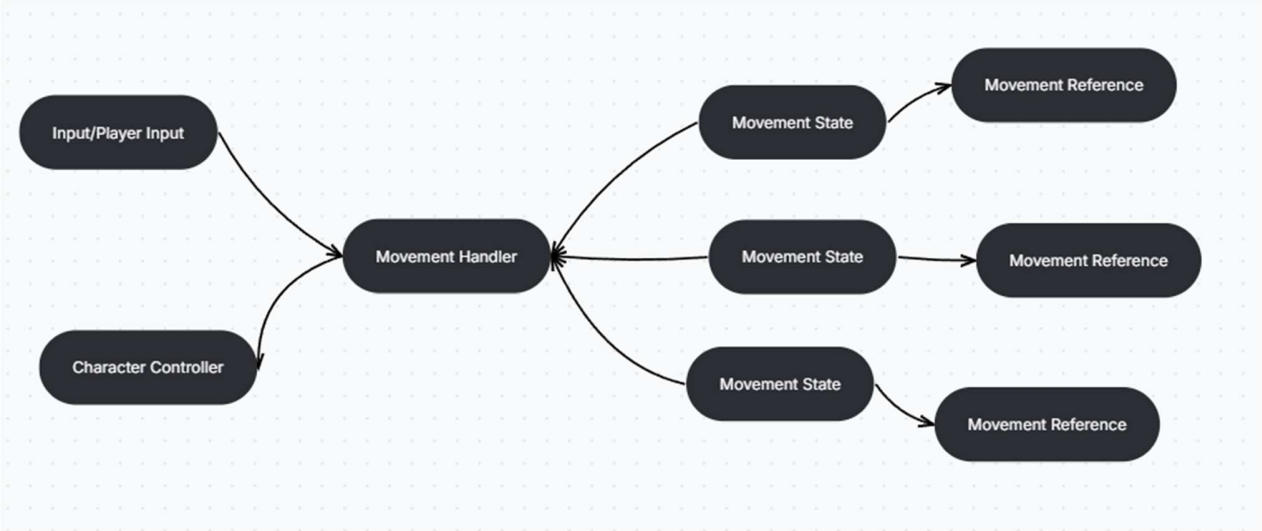


Figure 10: Representation of the Movement System's components.

The Input Handler will provide certain Get functions for the direction they wish to move in, or what actions like dashing or sprinting they wish to use. In this case, the player's keyboard Inputs will be used, but this could be swapped with a path-finding system that would instead provide the direction an enemy wish to move in.

The Movement States all provide different functionalities, such as walking, stunned, sprinting etc. Each state will oversee handling the velocity changes, listening to inputs, and transitioning to different states. The Movement Handler will provide these States the means to do so, such as change state and move functions. Each Movement State will also have access to a personal "Movement Reference", a resource that holds data such as max speed or acceleration. These can be swapped to make the movement act differently.

5. Implementation

5.1. Gameplay Loop and UI

The first thing I focused on when developing Going Up was the skeletons of the final gameplay loop. To do this, I created the four main game states, along with the functions to transition between them all.

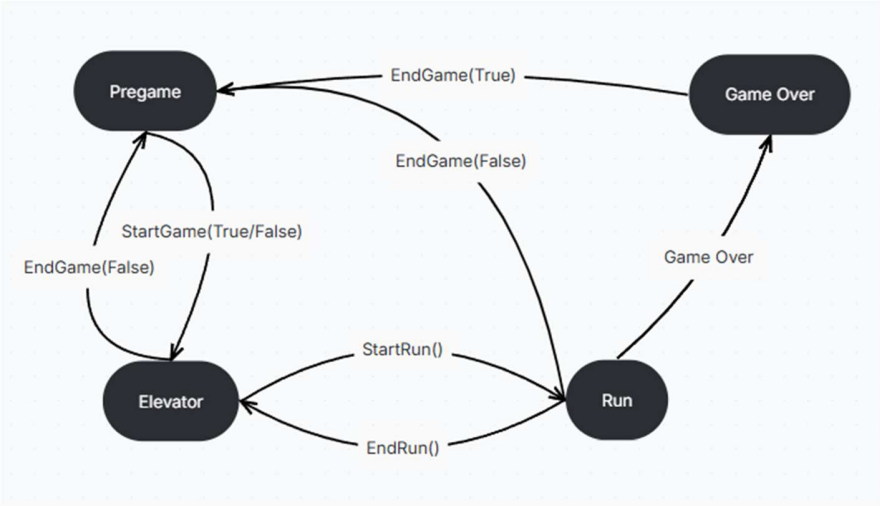
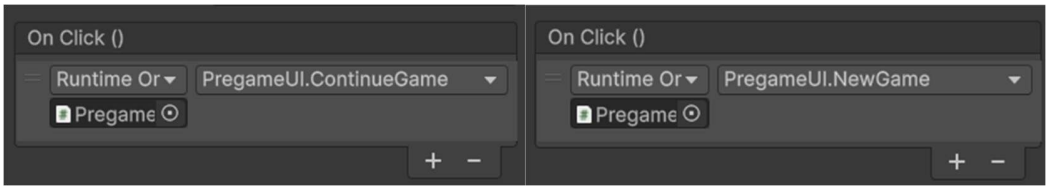


Figure 11: Updated Graph showing Game States.

Above is an updated diagram showing the functions used to transition between each state. Each function is run by connecting unity button events to a script for each UI page that runs each respective function. Each of these function names represents a function in the Game Manager script. Some of the functions (i.e. End Game and Start Game) take a Boolean argument, which is used to signal if the save file should be interacted with. If the save file is not to be used, defaults are loaded, or the data is not saved.

The snippets below show how the UI for the Pregame functions. Both the Continue and New Game buttons connect to a function within the Pregame UI script. These functions then call the Start Game function within the Game Manager script.




```
// pregame menu -> first elevator
2 references
public void StartGame(bool loadFromFile)
{
    if (loadFromFile)
    {
        gameStatus.LoadFromSave(false);
    } else
    {
        gameStatus.LoadFromSave(true);
    }
    ChangeState(Utils.GameStates.Elevator);
}
```

```
0 references
public void ContinueGame()
{
    gameManager.StartGame(true); // start game and load the save file
}

0 references
public void NewGame()
{
    gameManager.StartGame(false); // start game and load generic
}
```

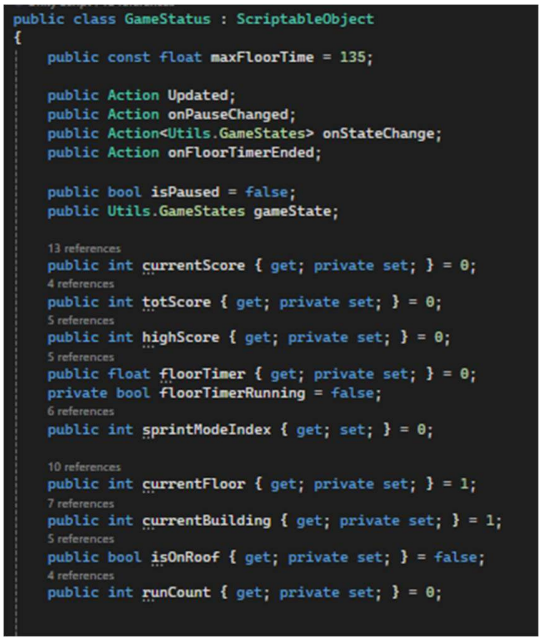
Figure 12, 13: screenshots of unity Buttons connected to the functions in figure 15

Figure 14: code snippet showing the Start Game function

Figure 15: code snippet showing the functions connecting the buttons to the Start Game function

The Game Manager is where all the transition functions are held. This means that no UI elements must individually run a transition, and the function can be reused. Ideally, no buttons are connected directly to this either, so having a script to middle the connection makes finding the source of each transition far easier, making debugging quicker.

The next step is to then have each element react to the changes made to the game state. This is done using the Game Status scriptable object [28]. I chose to use a scriptable object as at the time of development as I didn't fully understand the concept of a singleton. I chose this method as each script that needed to access it just needed a serialize field, and then I could drag and drop the scriptable object. Once I had understood singletons however, it was far too late to change it as it was integrated into many other scripts already.



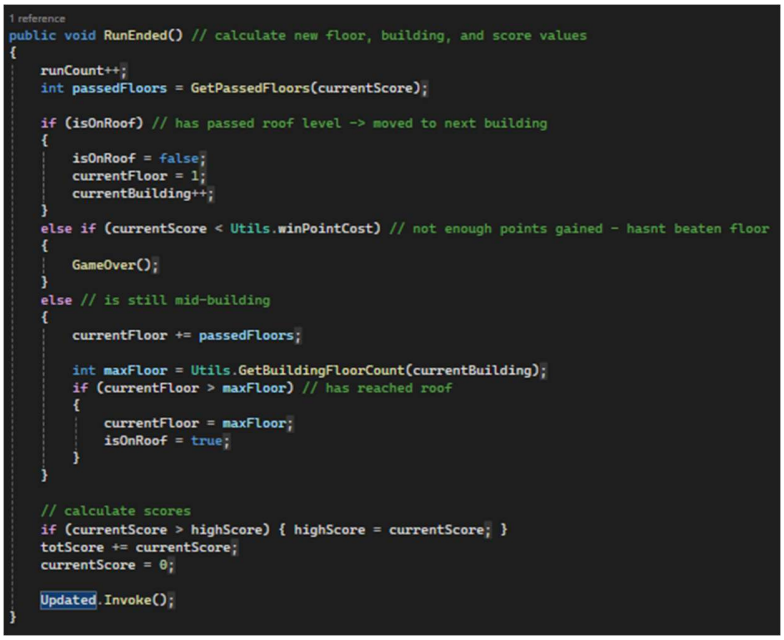
```
public class GameStatus : ScriptableObject
{
    public const float maxFloorTime = 135;

    public Action Updated;
    public Action onPauseChanged;
    public Action<Utils.GameStates> onStateChange;
    public Action onFloorTimerEnded;

    public bool isPaused = false;
    public Utils.GameStates gameState;

    13 references
    public int currentScore { get; private set; } = 0;
    4 references
    public int totScore { get; private set; } = 0;
    5 references
    public int highScore { get; private set; } = 0;
    5 references
    public float floorTimer { get; private set; } = 0;
    private bool floorTimerRunning = false;
    6 references
    public int sprintModeIndex { get; set; } = 0;

    10 references
    public int currentFloor { get; private set; } = 1;
    7 references
    public int currentBuilding { get; private set; } = 1;
    5 references
    public bool isOnRoof { get; private set; } = false;
    4 references
    public int runCount { get; private set; } = 0;
```

Figure 16: Code Snippet showing the Game Status Script Object

The Game Status Script object is used to save and provide the primary data needed for the game itself. Mainly, this contains the current score, floor and building number, floor timer, etc. each of these values have been set up to be privately set to make sure that the data remains consistent. To allow other scripts to then change these values, functions are set up to allow control over them.

Another function of the Game Status is to provide actions that other scripts can connect to. Examples of this are the Updated action that is run every time the data has been changed, or the on State Changed that runs every time the game state has transitioned.

The Load From Save function will either re-load select data from a save file or to a beginning state (Building 1 Floor 1). The Add Score function takes an integer argument and adds it to the current score and runs the Updated action. The Decrement Timer function will be called every frame and will decrease the run timer by the delta time and then run the GameOver function if it reaches 0.



```
1 reference
public void RunEnded() // calculate new floor, building, and score values
{
    runCount++;
    int passedFloors = GetPassedFloors(currentScore);

    if (isOnRoof) // has passed roof level -> moved to next building
    {
        isOnRoof = false;
        currentFloor = 1;
        currentBuilding++;
    }
    else if (currentScore < Utils.winPointCost) // not enough points gained - hasnt beaten floor
    {
        GameOver();
    }
    else // is still mid-building
    {
        currentFloor += passedFloors;

        int maxFloor = Utils.GetBuildingFloorCount(currentBuilding);
        if (currentFloor > maxFloor) // has reached roof
        {
            currentFloor = maxFloor;
            isOnRoof = true;
        }
    }

    // calculate scores
    if (currentScore > highScore) { highScore = currentScore; }
    totScore += currentScore;
    currentScore = 0;

    Updated.Invoke();
}
```

Figure 17: Code Snippet showing the Run Ended function

Above is a snippet of the Run Ended function that will calculate the game data once a run has ended. The is on Roof variable is used to flag if the player has reached the end of their current building. This function makes sure that the player has made enough points to move onto the next floor and then calculates how many floors they will go up.

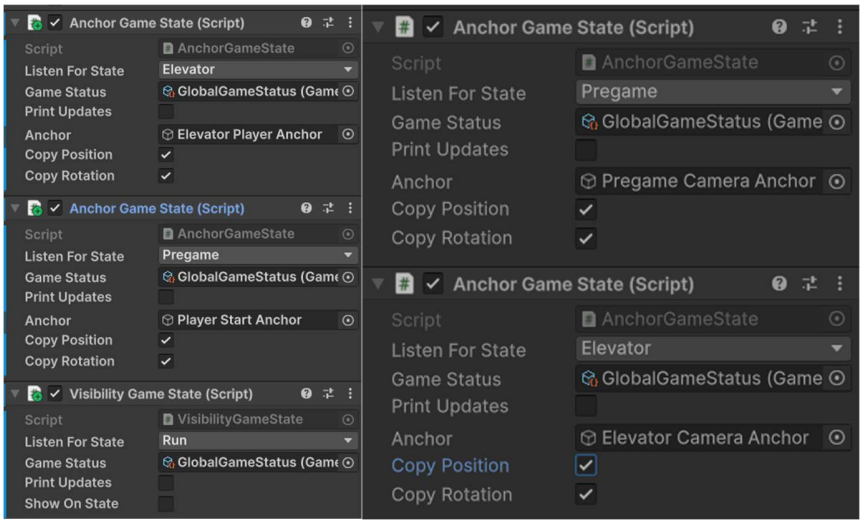
```
Unity Script | 5 references
public class GameState : MonoBehaviour
{
    [SerializeField] private Utils.GameStates listenForState;
    8 references
    public bool IsActive { get; private set; }
    public GameStatus gameStatus;
    public bool printUpdates;

    Unity Message | 6 references
    public virtual void Start()
    {
        gameStatus.onStateChange += OnStateChanged;
        OnStateChanged(Utils.GameStates.Pregame);
    }

    4 references
    public virtual void OnStateChanged(Utils.GameStates newState)
    {
        IsActive = newState == listenForState;
        if (printUpdates) { Debug.Log("State update - " + IsActive.ToString() + ". Im listening for " + listenForState.ToString()); }
```

Figure 19: Code Snippet showing the Game State Component

Finally, the UI uses the Game Status' actions to react to when the game state changes. To do this, I added the Game State Component. This is a framework that provides behaviours that react to game state changes.

The Anchor Game State is used to "anchor" an Objects position and rotation to another Game Object's transform. An example of this is the Player Puppet (a duplicate of the player

model without the behaviour) used in the UIs – between different states, it's located in different areas. The Visibility Game State is used to either make an object visible or invisible when a certain game state is active. This is used primarily for the UI, as each page uses this to show when its respective game state is active.



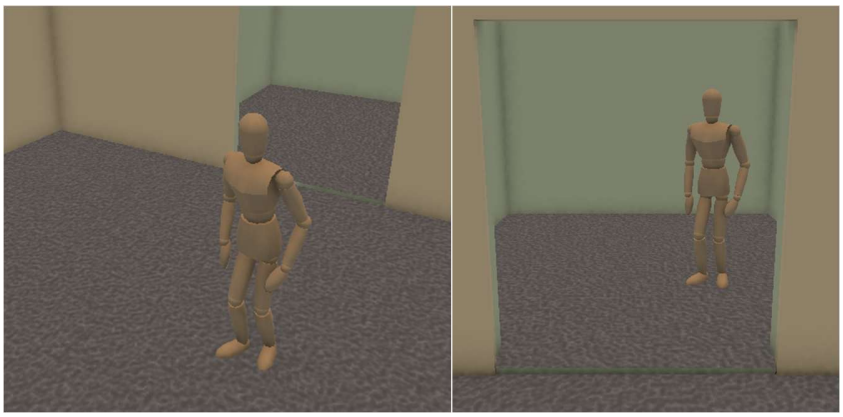


Figure 20: Screenshot of the Anchor and Visibility game states on the Player Puppet

Figure 21: Screenshot of the Anchor game states on the Camera

Figure 22, 23: Screenshots of the Player Puppet and Camera during the Pregame and Elevator game states

5.2. Save File

To create the save file, I am using two objects: the Save Manager and Save Data. The Save Manager is used to transfer the Game Status' data into a binary (serializable) format; The Save Data object is made to represent this data, and will either copy it from the Game Status, or load generic data.

```
[System.Serializable]
// this handles the data in the save file, i.e. formatting it
// or giving it to the variables that need it, IE PlayerData
// loading or saving the data itself is done seperately by
// the SaveManager
14 references
public class SaveData
{
    public int totScore = 0;
    public int highScore = 0;
    public int currentFloor = 1;
    public int currentBuilding = 1;
    public bool isOnRoof = false;
    public int runCount = 0;
    public int sprintModeIndex;
```

```
1 reference
public void loadFromGameStatus(GameStatus status)
{
    totScore = status.totScore;
    highScore = status.highScore;
    currentFloor = status.currentFloor;
    currentBuilding = status.currentBuilding;
    isOnRoof= status.isOnRoof;
    runCount = status.runCount;
    sprintModeIndex = status.sprintModeIndex;
}

3 references
public void LoadGeneric()
{
    totScore = 0;
    highScore = 0;
    currentFloor = 1;
    currentBuilding = 1;
    isOnRoof = false;
    runCount = 0;
    sprintModeIndex = 0;
}
}
```

Figure 24, 25: Snippet of the Save Data's attributes and functions

Since the class is entirely static, any class can run save and load operations when needed without requiring a pointer. To save, the Game Status is passed to a new Save Data class to copy the needed data and then written to the file location. To load, it checks if loads the save file; If it is not valid, a new generic Save Data object is created. This Save Data is then copied to the Game Status and saved if the file was not valid previously to ensure the save file exists for later.

5.3. Player Movement System

5.3.1. Overall Movement System

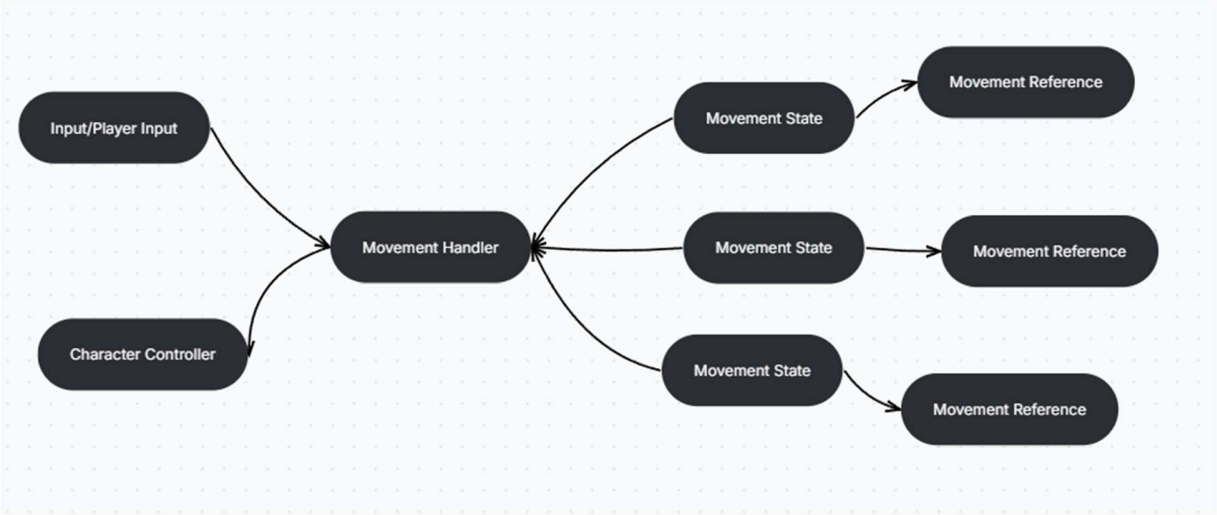


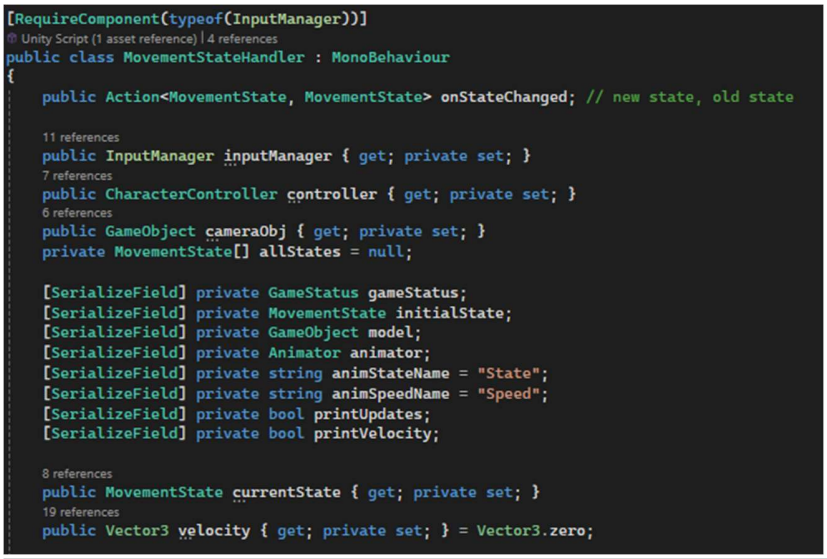
Figure 10: Representation of the Movement System's components.

As described in 4.3.1, the Movement System relies on multiple different components working in harmony. Above is the diagram used, which still holds up with the final rendition of the system – if you are not familiar with this now, I recommend going back.

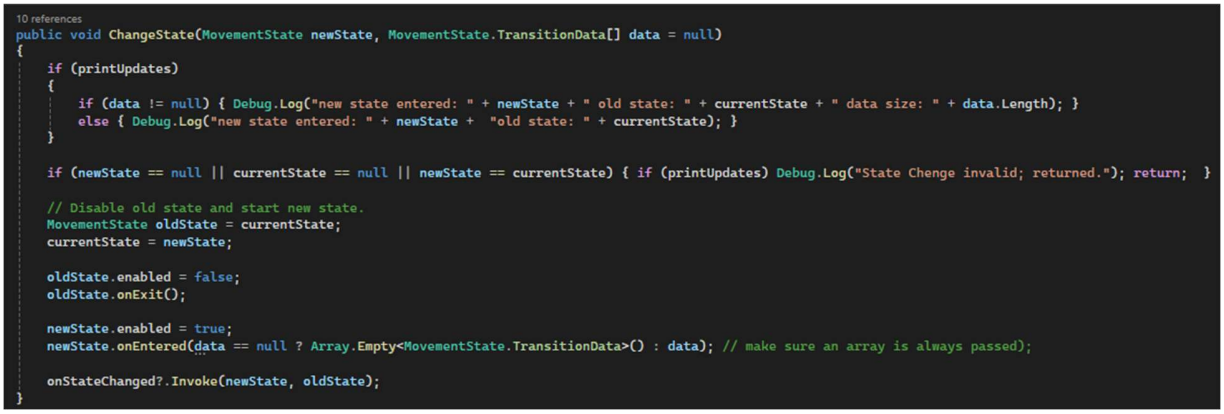
```
5 references
public interface InputManager
{
    2 references
    public Vector3 GetCurrentInput(); // directional movement, such as WASD for the player
    3 references
    public bool GetWishSprint(); // can be implemented for other enemies but ideally only for player
    2 references
    public bool GetWishJump(); // ditto
    4 references
    public bool GetWishDash(); // ditto

}
```

Figure 26: Code Snippet of the Input Manager Component

The smallest system of them all was the Input Manager. This script provides 4 functions that will be accessed by movement states. The only iteration of this is the Player Input, which simply overrides these functions and returns the keyboard inputs from the player.



```
[RequireComponent(typeof(InputManager))]
Unity Script (1 asset reference) | 4 references
public class MovementStateHandler : MonoBehaviour
{
    public Action<MovementState, MovementState> onStateChanged; // new state, old state

    11 references
    public InputManager inputManager { get; private set; }
    7 references
    public CharacterController controller { get; private set; }
    6 references
    public GameObject cameraObj { get; private set; }
    private MovementState[] allStates = null;

    [SerializeField] private GameStatus gameStatus;
    [SerializeField] private MovementState initialState;
    [SerializeField] private GameObject model;
    [SerializeField] private Animator animator;
    [SerializeField] private string animStateName = "State";
    [SerializeField] private string animSpeedName = "Speed";
    [SerializeField] private bool printUpdates;
    [SerializeField] private bool printVelocity;

    8 references
    public MovementState currentState { get; private set; }
    19 references
    public Vector3 velocity { get; private set; } = Vector3.zero;
```



```
10 references
public void ChangeState(MovementState newState, MovementState.TransitionData[] data = null)
{
    if (printUpdates)
    {
        if (data != null) { Debug.Log("new state entered: " + newState + " old state: " + currentState + " data size: " + data.Length); }
        else { Debug.Log("new state entered: " + newState +  "old state: " + currentState); }
    }

    if (newState == null || currentState == null || newState == currentState) { if (printUpdates) Debug.Log("State Chenge invalid; returned."); return;  }

    // Disable old state and start new state.
    MovementState oldState = currentState;
    currentState = newState;

    oldState.enabled = false;
    oldState.onExit();

    newState.enabled = true;
    newState.onEntered(data == null ? Array.Empty<MovementState.TransitionData>() : data); // make sure an array is always passed);

    onStateChanged?.Invoke(newState, oldState);
}
```

Figure 27: Code Snippet of the Movement State Handler's definition.

Figure 28: Code Snippet of the Movement State Handler's Change State Function

For the Movement Handler to work, it requires references to the Input Manager, Character Controller and animator. The Movement Handler then provides functions to control these objects such as updating animation states or changing the velocity of the Character Controller

The Movement Handler also handles transitions between Movement States; the Change State function will change the current state, run the on Exit and on Entered functions on the respective state, and then invoke the on State Changed Action. The Transition Data is an Array of flags that allows each Movement State to communicate transition behaviors, such as ignoring max speed once transitioned in.

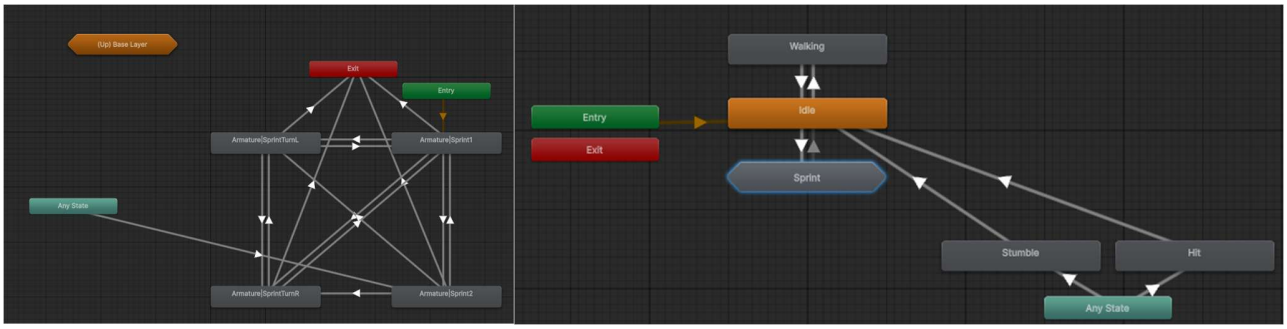


Figure 29, 30: The Animator graph for the player model – this is controlled via the Movement Handler using provided arguments.

```
public abstract class MovementState : MonoBehaviour
{
    22 references
    public enum TransitionData
    {
        Force, IgnoreVelocityCap, IgnoreCoyoteTime, KeepCamRotationMode, IgnoreStumbleTime, StartMach4
    }

    [SerializeField] public MovementStateReference reference;
    60 references
    public MovementStateHandler stateHandler { get; private set; }
```

Figure 31: Code Snippet of the Movement State Definition

The Movement State is defined very simply – it has an on Entered and on Exit function, and is provided with a Movement State Reference, a scriptable object [28] that holds a collection of variables that specify behaviors. I decided to do this rather than a collection of constants as it's easier to edit them at runtime, along with being able to swap them out without needing extra logic within the state.

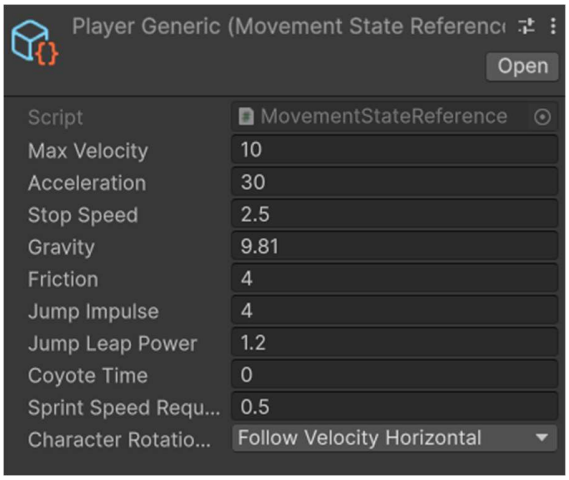
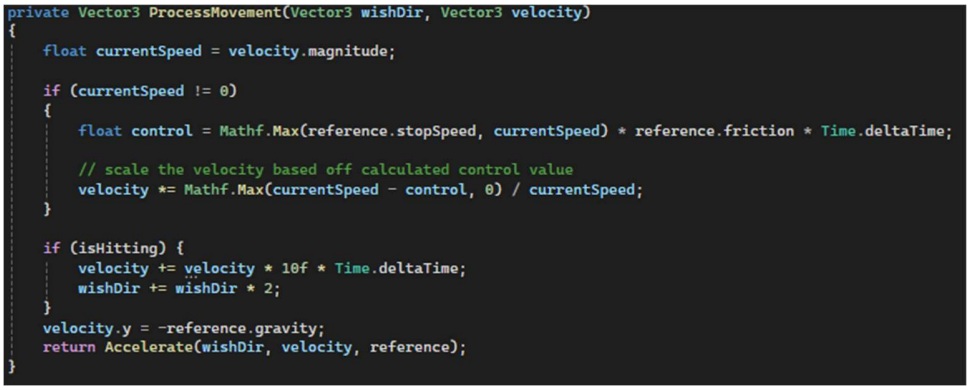
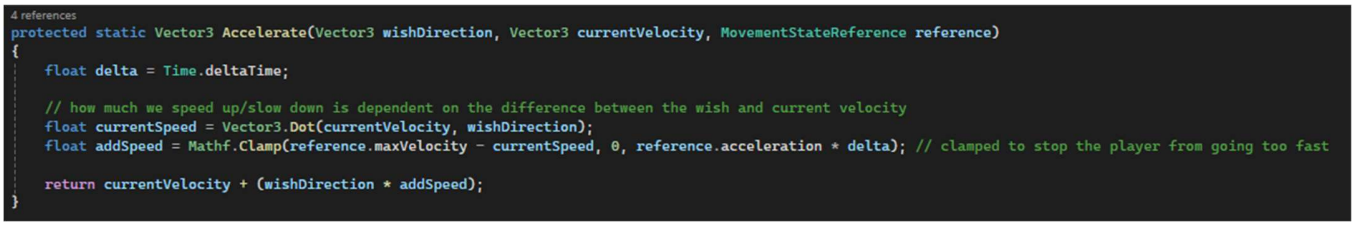


Figure 32: Screenshot of an Example Movement State Reference used for player walking



```
private Vector3 ProcessMovement(Vector3 wishDir, Vector3 velocity)
{
    float currentSpeed = velocity.magnitude;

    if (currentSpeed != 0)
    {
        float control = Mathf.Max(reference.stopSpeed, currentSpeed) * reference.friction * Time.deltaTime;

        // scale the velocity based off calculated control value
        velocity *= Mathf.Max(currentSpeed - control, 0) / currentSpeed;
    }

    if (isHitting) {
        velocity += velocity * 10f * Time.deltaTime;
        wishDir += wishDir * 2;
    }
    velocity.y = -reference.gravity;
    return Accelerate(wishDir, velocity, reference);
}
```



```
4 references
protected static Vector3 Accelerate(Vector3 wishDirection, Vector3 currentVelocity, MovementStateReference reference)
{
    float delta = Time.deltaTime;

    // how much we speed up/slow down is dependent on the difference between the wish and current velocity
    float currentSpeed = Vector3.Dot(currentVelocity, wishDirection);
    float addSpeed = Mathf.Clamp(reference.maxVelocity - currentSpeed, 0, reference.acceleration * delta); // clamped to stop the player from going too fast

    return currentVelocity + (wishDirection * addSpeed);
}
```

Figure 33: Code Snippet of the Move State Walk's Process Movement function.

Figure 34: Code Snippet of the Accelerate function.

Above are snippets showing the Move State Walk's Process Movement function, along with the Accelerate function within the Movement State. This system is directly inspired by the Quake movement engine [16][19] and works by scaling down the velocity by the "stop speed" and "friction" first, and then accelerating the movement based on the player's inputs.

Most of the Process Movement function is replicated between all the movement states. The is Hitting flag references a feature where once the player presses space while walking, they will play a short animation where the player can break objects as they would while sprinting (explained in 5.5).

```
Unity Message | 0 references
private void Update()
{
    // get input direction
    Vector3 wishDir = GetMoveDirection(stateHandler.inputManager, inputMapping, stateHandler.cameraObj);

    // -> ungrounded - allow coyote time
    if (notGroundedState != null && !stateHandler.controller.isGrounded) { stateHandler.ChangeState(notGroundedState); return; }

    // -> attack/hit
    if (stateHandler.inputManager.GetWishDash()) { Hit(); }
    HandleHit();

    // calculate velocities
    Vector3 oldVelocity = stateHandler.velocity;
    Vector3 newVelocity = ProcessMovement(wishDir, oldVelocity);
    float horizontalSpeed = Utils.GetHorizontal(newVelocity, false).magnitude;


    stateHandler.Move(newVelocity);
    stateHandler.SetAnimatorState(horizontalSpeed > 0.05? walkingAnimationState : idleAnimationState);
    stateHandler.SetAnimatorSpeed(horizontalSpeed * animationSpeedMultiplier);
    stateHandler.Rotate(reference.characterRotationMode);

    // -> sprint + jumping
    AttemptJump(reference, notGroundedState, jumpAnimationTrigger);
    AttemptSprint(reference, horizontalSpeed, onSprintState);
}
```

Figure 35: Snippet of the Move State Walk Update function. This shows all the steps in order. The "->" annotations show where transitions to different states are.

```
Unity Message | 0 references
private void Update()
{
    Vector3 velocity = stateHandler.velocity;
    Vector3 newVelocity = ProcessMovement(velocity, Vector3.zero);

    stateHandler.Move(newVelocity);
    // -> sprint (via dash move)
    if (stateHandler.inputManager.GetWishDash()) { stateHandler.ChangeState(sprintState, new TransitionData[] {TransitionData.StartMach4}); }

    // once slowed down, start reducing stumbleState time
    if (Utils.GetHorizontal(newVelocity, false).magnitude <= 0.1f)
    {
        stumbleTime -= Time.deltaTime;
        if (stumbleTime <= 0f) stateHandler.ChangeState(returnState); // once stumbleState time 0, return
    }
}
```

Figure 36: Snippet from the stumble move state, showing the different Update function.

5.4.2 Sprint system, dashing and sprint styles.

The sprint system is far more complex than the generic movement due to the controls. Below is a snippet of the Move State Sprint component's definition. For an explanation of how the sprint should work go to section 3.2.3.

```
public class MoveStateSprint : MovementState
{
    public MovementState walkState;
    public MovementState notGroundedState;
    public MovementState stumbleState;

    public SprintRefProvider sprintRef;
    [SerializeField] private int runningAnimationState = 2;
    [SerializeField] private string sprintingAnimStateName = "SprintState";
    [SerializeField] private float minSprintAnimSpeed = 1.5f;
    [SerializeField] private float animationSpeedMultiplier = 0.1f;
    [SerializeField] private string jumpAnimationTrigger = "";
    [SerializeField] private float maxDashTimer = 1f;
    [SerializeField] private float maxDashCooldown = 1f;

    private const int sprintAnimState = 1;
    private const int maxSprintAnimState = 2;
    private const int turnLeftAnimState = 3;
    private const int turnRightAnimState = 4;

    private const float bumpCastYOffset = 0.36f;
    private const float bumpCastDistance = 0.6f;

    private bool isDashing = false;
    private float dashTimer = 0f;
    private float dashCooldown = 0f;
    private Vector3 dashDirection;

    private int currentMach = 2;
    private Vector3 input;
    private bool isTurning = false;
    private int turnAnim = -1;
    0 references
    private enum TurnDirections { right, left, backward };
```

Figure 37: Code Snippet of the Move State Sprint definition.

The main difference between the sprint and other movement modes is the use of the Sprint Reference Provider. This is a Scriptable Object [28] that provides multiple Movement References. This is needed since the Sprint Movement State requires multiple different Move States References for each mach.

The “turning” of the sprint is done by checking if the input of the player is different enough from the current velocity by using a Dot product [18]. If the player is turning left or right, the animation is changed to show this. If the player is turning in the opposite direction (dot of 0.85f), the player will stumble. During the turn, a turning Movement Reference will be used to slow down the player by increasing the friction and stop speed and decreasing the acceleration and max speed.

```
9 references
private MovementStateReference GetMachRef(int mach = -1) // uses the default "reference" if one isnt selected, otherwise selects from the sprintRef
{
    if (isDashing) return sprintRef.Dash == null ? reference : sprintRef.Dash;
    if (isTurning) return sprintRef.Turning == null ? reference : sprintRef.Turning;
    if (currentMach == -1) return reference;

    switch (mach)
    {
        case -1: return GetMachRef(currentMach);
        case 2: return sprintRef.Mach2;
        case 3: return sprintRef.Mach3;
        case 4: return sprintRef.Mach4;
    }
    return reference;
}
```

```
1 reference
private int ProcessMach(float speed)
{
    // mach up
    if (currentMach < 4 && !isTurning)
    {
        MovementStateReference nextMach = GetMachRef(currentMach + 1);
        if (speed > nextMach.sprintSpeedRequirement) { return currentMach + 1; }
    }

    // mach down
    if (currentMach > 2)
    {
        if (speed < GetMachRef().sprintSpeedRequirement) { return currentMach - 1; }
    }

    return currentMach;
```

Figure 38: Code Snippet of the Get Mach Reference function

Figure 39: Code Snippet of the Process Mach function.

Above is the code controlling mach. As the player's speed increases, the mach increases too, changing the Movement Reference used. The Process Mach function is run every frame to ensure that the mach stays consistent with the player's speed.

Dashing is activated by pressing the spacebar while sprinting. This is deactivated after a timer, or if the player presses space again. During this state, Process Dash function is called, which will call the Process Movement function with the direction the dash was started in, rather than player input. This means that the player cannot control the movement when dashing until the timer ends, or they press the space key again.

```
1 reference
private bool ProcessBump(Vector3 forward)
{
    RaycastHit hit;
    Physics.Raycast(transform.position + new Vector3(0, bumpCastYOffset, 0), forward, out hit, bumpCastDistance);

    return hit.collider != null;
}
```

Figure 40: Code Snippet of the Process Bump function

Every frame, a Raycast [29] is used to see if the player has collided with anything in front of them. If this cast does hit a collider, the player is sent into a stumble.

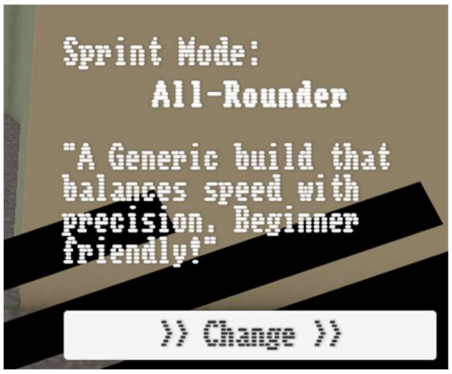


Figure 41: Screenshot of the Sprint mode section in the Elevator UI

In the elevator UI, the player is shown a button that can swap the "movement style". This will change the movement style index in the save file and replace the sprint reference.

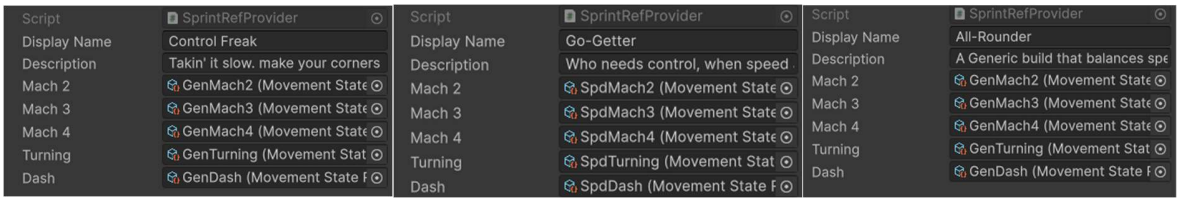


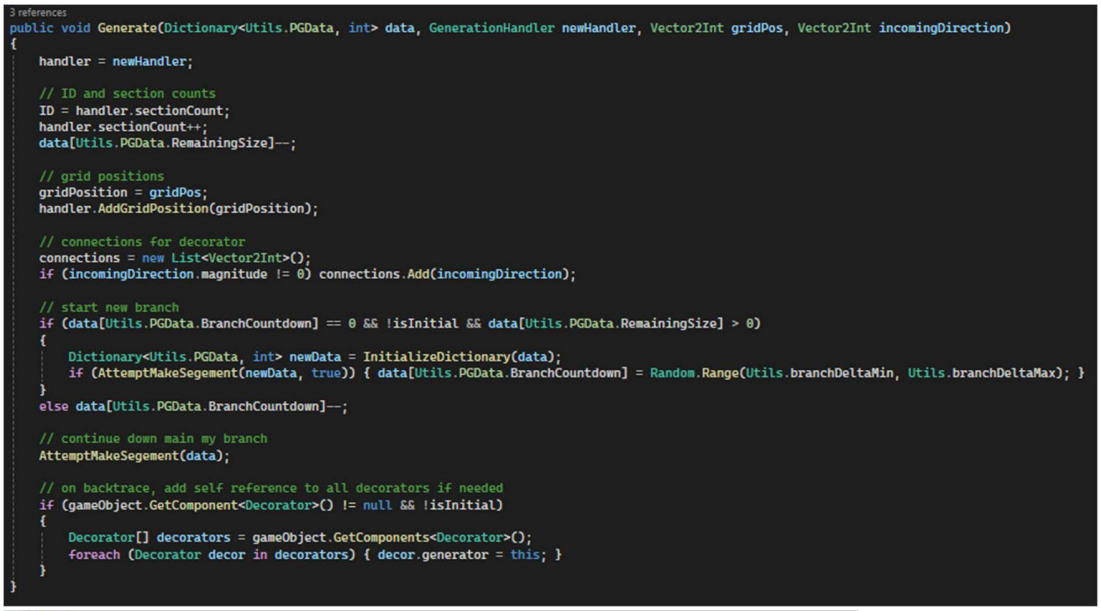
Figure 42, 43, 44: Screenshots of the 3 Sprint References, including their names.

5.4. Procedural Generation

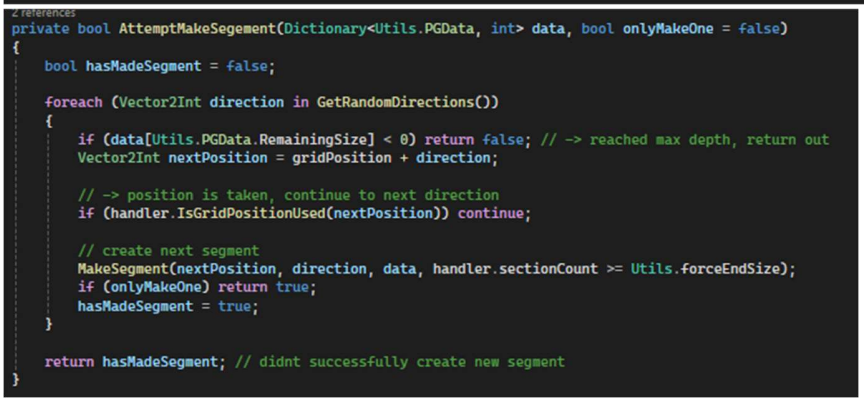
```
static public int GetFloorSeed(int floor, int building, bool isNegative = false)
{
    // match both sides of the seed to the correct sizes
    string seedPrefix = seedPadding + building.ToString();
    if (seedPrefix.Length > seedPrefixSize) seedPrefix = seedPrefix.Substring(seedPrefix.Length - seedPrefixSize);

    string seedSuffix = seedPadding + floor.ToString();
    if (seedSuffix.Length > seedSuffixSize) seedSuffix = seedSuffix.Substring(seedSuffix.Length - seedSuffixSize);

    int result = int.Parse(seedPrefix + seedSuffix);
    return isNegative ? -result : result;
}
```

Figure 45: Code Snippet of the Get Seed function.

For each floor, the seed needs to be generated first. This is done by taking the floor and building number and putting them together in a way that will cause as little repeat as possible (snippet above). The map generation is then started by the Generation Handler script by spawning in map sections called Generation Object (GenObj), along with randomizing the color of the props. The GenObj will then call the next GenObj, creating recursion. To control this, a dictionary of values is passed to every new part as it goes down. This contains the seed, remaining branch size, branch countdown, and is main branch flag. Every new section will decrease the remaining branch size and branch countdown by one.



```
3 references
public void Generate(Dictionary<Utils.PGData, int> data, GenerationHandler newHandler, Vector2Int gridPos, Vector2Int incomingDirection)
{
    handler = newHandler;

    // ID and section counts
    ID = handler.sectionCount;
    handler.sectionCount++;
    data[Utils.PGData.RemainingSize]--;

    // grid positions
    gridPosition = gridPos;
    handler.AddGridPosition(gridPosition);

    // connections for decorator
    connections = new List<Vector2Int>();
    if (incomingDirection.magnitude != 0) connections.Add(incomingDirection);

    // start new branch
    if (data[Utils.PGData.BranchCountdown] == 0 && !isInitial && data[Utils.PGData.RemainingSize] > 0)
    {
        Dictionary<Utils.PGData, int> newData = InitializeDictionary(data);
        if (AttemptMakeSegement(newData, true)) { data[Utils.PGData.BranchCountdown] = Random.Range(Utils.branchDeltaMin, Utils.branchDeltaMax); }
    }
    else data[Utils.PGData.BranchCountdown]--;

    // continue down main my branch
    AttemptMakeSegement(data);

    // on backtrace, add self reference to all decorators if needed
    if (gameObject.GetComponent<Decorator>() != null && !isInitial)
    {
        Decorator[] decorators = gameObject.GetComponents<Decorator>();
        foreach (Decorator decor in decorators) { decor.generator = this; }
    }
}
```



```
2 references
private bool AttemptMakeSegement(Dictionary<Utils.PGData, int> data, bool onlyMakeOne = false)
{
    bool hasMadeSegment = false;

    foreach (Vector2Int direction in GetRandomDirections())
    {
        if (data[Utils.PGData.RemainingSize] < 0) return false; // -> reached max depth, return out
        Vector2Int nextPosition = gridPosition + direction;

        // -> position is taken, continue to next direction
        if (handler.IsGridPositionUsed(nextPosition)) continue;

        // create next segment
        MakeSegment(nextPosition, direction, data, handler.sectionCount >= Utils.forceEndSize);
        if (onlyMakeOne) return true;
        hasMadeSegment = true;
    }

    return hasMadeSegment; // didnt successfully create new segment
}
```

Figure 42: Code Snippet of the Generation Handler's Generate function.

Figure 43: Code Snippet of the Attempt Make Segment function.

If the branch countdown reaches 0, a new dictionary is created. This will have a smaller remaining size counter, a negative branch countdown, and the is main branch flag set to false. This will then generate a sub-branch that once at its max size, will return to this section, where it will continue. Next, the Gen object will attempt to make another segment. If this section has no unoccupied directions it can move in, or the remaining size counter is at 0, this will fail.

Next, each room runs its Decorator component. There are two main decorators: halls and large offices. The Corner Decorator will check all four directions, and either place a wall or hallway depending on if there is a connection defined by the GenObj or not. The Large Office Decorator will fill out the entire tile it starts attempts grow in one random direction. It will

place walls and one door that connects it to the hall. Each of these sections will also contain Decor Spawner.

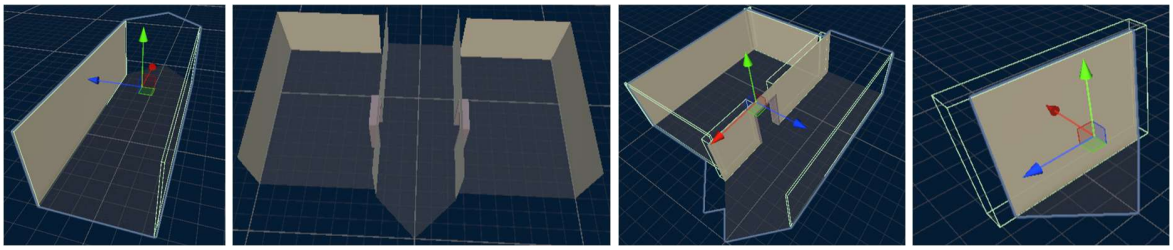


Figure 44 – 47: Screenshots of the Different models made in ProBuilder used to create each section of hall

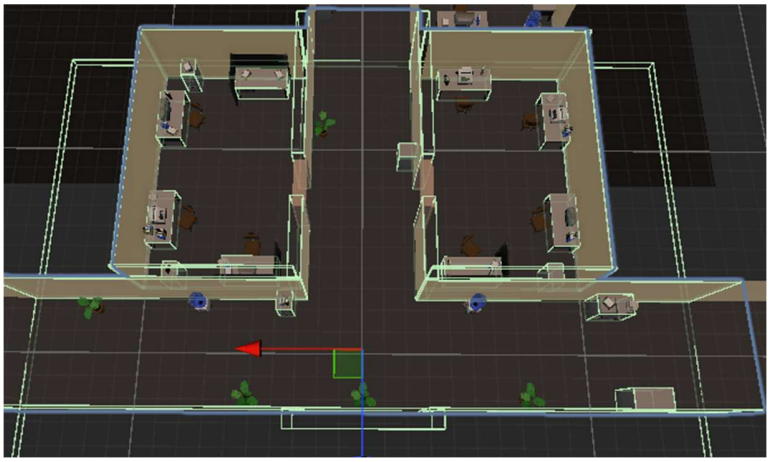


Figure 48: Screenshot of one completed hall section – props included.

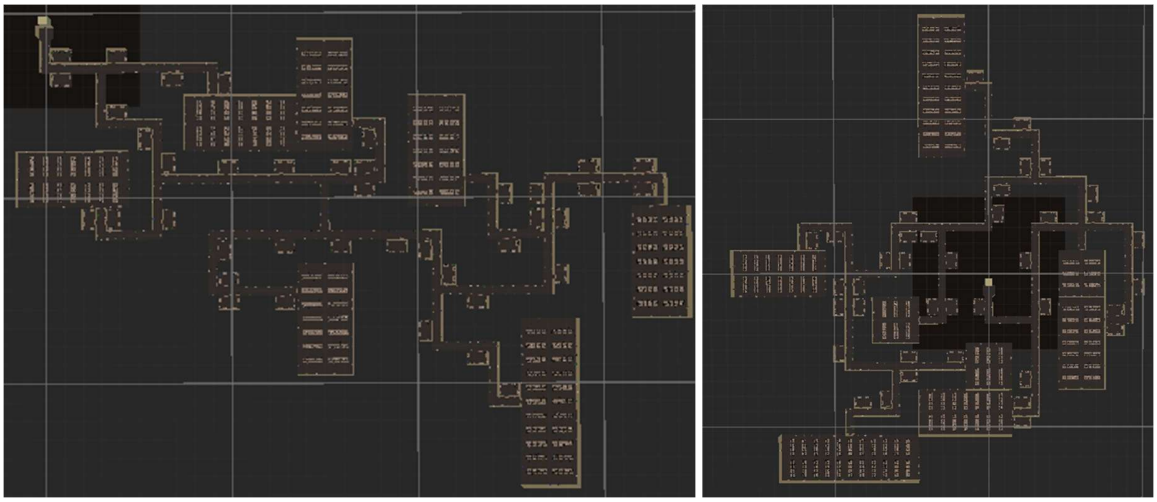


Figure 49 – 52: Screenshots of 4 random completed layouts.

5.5. Props and destruction system

Each prop is first generated once the layout is finished (see 5.4.). Every section spawn with Decor Spawners, which will have a set tag, allowing the engine to return an array of all the spawners. Each Decor Spawner will also have an array of props that it can spawn to make sure that no props spawn where they shouldn't be. The Generation Handler will run all these spawners that will replace itself with a prop. This will then run until all the Decor spawners have replaced themselves. This must be done using a while loop, as Decor Spawners can spawn embedded inside props.

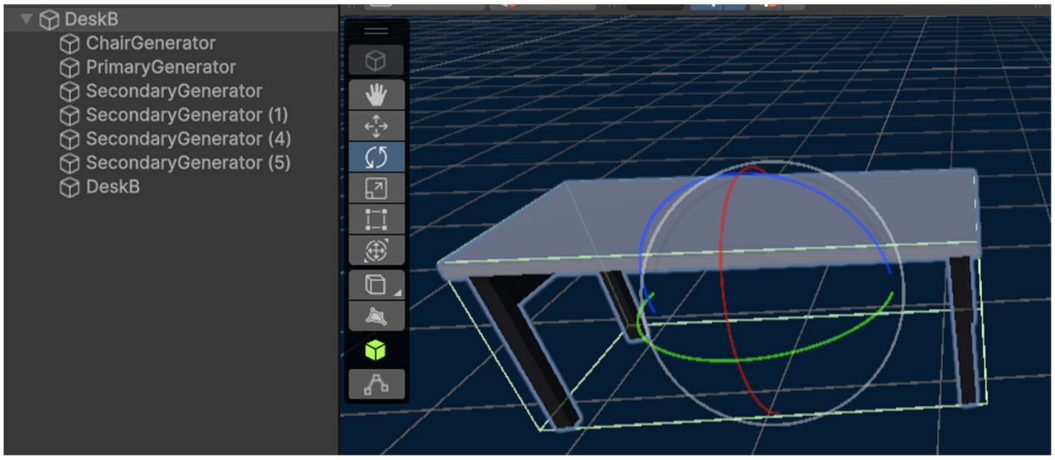


Figure 53: Screenshot of a Desk Prop inside of Unity Editor – shows the embedded Decor Spawners (Primary and secondary generators).

Figure 54: Desks generated within a Small Office, with props generated on top or beside it.

```
5 references
public class Prop : MonoBehaviour
{
    [SerializeField] private MapColourHandler colourHandler;
    [SerializeField] private GameObject[] models;
    [SerializeField] private bool doChangeMaterial;
    [SerializeField][Range(0f, 360f)] private float allowedSpawnRotation = 0;
    [SerializeField] private bool canBreak = true;
    [SerializeField] private int breakStrength;
    [SerializeField] private int breakPoints;
    [SerializeField] private AudioClip breakSFX = null;
    [SerializeField] private float breakSFXVolume;
    [SerializeField] private GameStatus gameStatus;
```

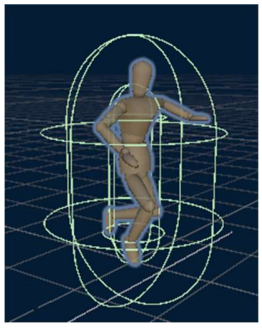


Figure 55: Code Snippet of the Prop Component definition.

Figure 56: Screenshot of the Player's Capsule Collider.

Each `Prop` has settings for its generation along with controls for its breakability. This includes the mach the player must meet to break it, the points provided, and the SFX it should play when broken.

The player has a collider attached to it that will check every `Prop` that enters it. If the player has a high enough mach, it will call the `Destroy` function on the `Prop`. This will add the points to the score and run a one-shot sound effect. If the player is sprinting, the sprint's mach is used. If the player is walking, the player can press space to use mach 4 damage temporarily.

5.6. Assets and Editing

All of props for this game were existing assets (see asset credits – 9.2) that I had adjusted. I made all the props have a lower polygon count and then redid the UVs to fit my textures. Each of these were then exported as one "GLB" file and then made into separate prefabs.

Figure 57 – 60: Screenshots of the Props laid out in the Editor along with their gradient textures.

The textures are set up in a way that there are multiple gradients, with 3 colors being interchangable. Each props UV is fit onto these gradients, and once the texture is swapped, the color on the props will change too. This texture is updated at the start of every Map Generation cycle, and is dictated by the seed.

Figure 58: The original props and textures – as seen on sketchfab

The player character itself has also been edited to have a simpler texture, and new animations.

6. Testing

6.1. System Testing

While implementing the save file and the procedural generation system, I found that testing them exclusively by running the game became restrictive, since running and then stopping unity takes quite a lot of time. To help me debug and test my code, I built tools that I could dock directly in unity using the built-in unity tools system.

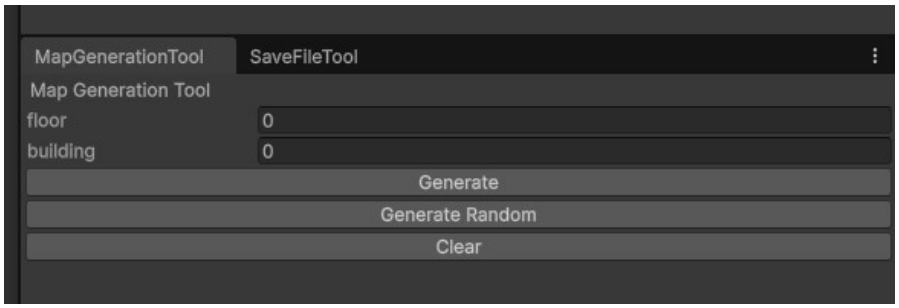
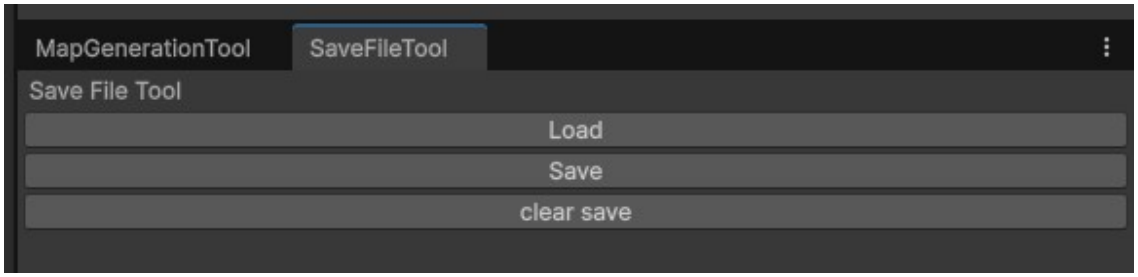


Figure 59: Screenshot of the map Generation Tool.

For the procedural generation tool (Figure 59), I provided myself with 3 main functions: two generation buttons (one that runs off a set seed, and another that runs off a random seed), and a clear map button. This meant that I could run the procedural generation system at any point I wanted and use the unity editor to debug Generation Objects and Props or use the camera to view the map from a birds-eye view.

I made another tool that provided me with buttons to load, save and clear the current save file. To use this tool, I must load the save file which will allow me access to text-edit fields that I can use to save custom values to my own save file. This allowed me to check that the save file stayed persistent after closing the game and directly edit my save file for later testing.



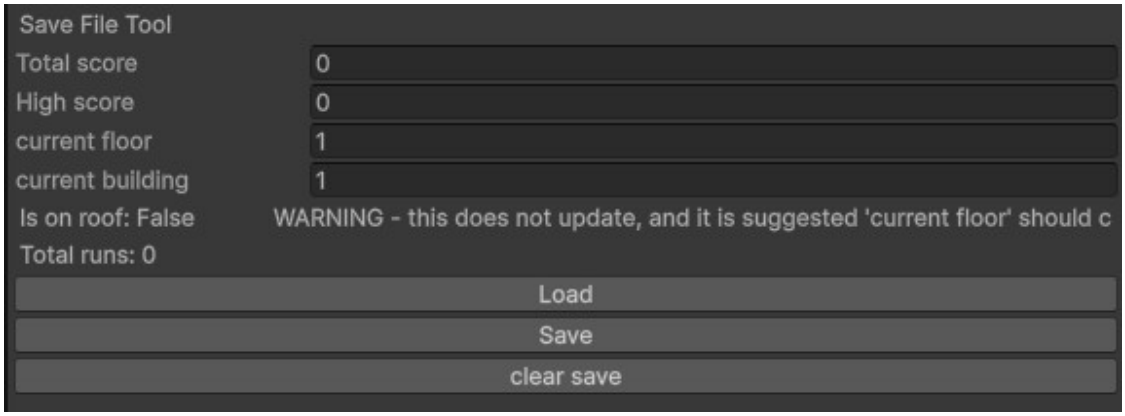


Figure 60: Screenshot of the Save File tool inside the dock before loading the save file.

Figure 61: Screenshot of the Save File tool after loading the save file. In this case, there is an empty Save File.

6.2. User Testing and feedback – first wave

On the 3rd of March, I produced a working prototype of the game with 2 of the 3 core features being developed, with the third (gameplay customization/sprint mode selection) not being included fully at this time. I set this testing with 3 main focuses: finding bugs I had missed in system testing, seeing how people interacted with the movement system and level generation, and finally, I asked each participant what feature they would want me to add next: enemies to populate the levels, or movement upgrades.

I had 6 testers who all ran the game separately, each gave verbal consent for their responses to be recorded after the script was read out, and after testing had concluded.

Tester #	Response to movement system	Response to Map generation	Overall Enjoyability	Enemies or Upgrades
1	*Enjoyed the responsiveness of the player animations. Didn't struggle too much with movement and consistently reached enough points to move to the next level. Struggles with being stunned after sprint-dashing.*	*Preferred large offices to smaller offices. Did explore further than first large office to find a second unless timer was low. Did struggle with backtracking new levels.*	*They enjoyed the concept of the game. Only feedback was no use to small offices.*	*Both, but preference for Upgrades.*
2	*Enjoyed pushing movement system to the limit – attempting to find "Tech". did find that movement veers to 8axis with WASD, maybe "use mouse to steer".*	*Requested small offices be larger along with all the doors. Mentioned having no incentive to attempt the smaller offices.*	*Overall enjoyed the game but requested more feedback on actions such as breaking objects.*	*Upgrades – dislikes the idea of enemies intervening with movement*
3	*Struggles with hitting walls and office doors. Also enjoyed pushing movement system to its limit. Attempted to use movement system to leave the map (unsuccessfully)*	*More incentive requested for smaller offices, no reason to access them. "Large offices are too overpowered" as they offer too many points.*	*Enjoyed the movement primarily. Ended up spending testing not attempting to beat levels but instead playing with movement system.*	*Upgrades*
4	*Disliked the camera angle as they "couldn't see what's coming". Did struggle with hitting walls due to camera.*	*Break speed requirement of hallway props was "inconsistent".*	*Due to difficulty adjusting to camera angle they didn't test for long but did enjoy the concept of the game.*	*Upgrades, but no real preference.*
5	*They enjoyed the movement as is but requested a way to "cancel the dash by pressing the button again" Also requested a speed-gauge as part of the UI, along with a boost when breaking objects.*	*They didn't use small offices often and eventually found a consistent method in larger offices.*	*Could not fully interact with the game due to monitor sizing affecting UI usability (see bugs). Also requested a music-mute button*	*Upgrades – pushed for one-time abilities as part of this.*
6	*Mentioned that walking is slow to the point its unusable. Mentioned a way to utilize the stun after hitting a wall. They wanted particle effects for breaking props. When asked, mentioned the camera angle was fine as-is.*	*Requested better colliders for walls as kept skimming doorframes or walls. Said that the point difference for different objects should be clearer.*	*Overall enjoyed the game but felt that the SFX left something to be desired. Felt the movement system was unique and enjoyable.*	*Enemies – no real difficulty yet, add upgrades as a response to enemies.*

Table 1: User testing 1 responses.

The overall response was positive – 5 of the 6 testers found the game accessible, while the last found the camera angle a pain point. The movement system was a success, as all the users eventually found themselves enjoying it, some saying it was akin to drifting in Mario kart when turning. Two of the big issues I saw, however, was that everyone struggled to avoid hitting walls while sprinting, and favored the larger offices over smaller ones, finding the latter difficult to traverse due to their size. A lot of bugs were also found during testing, including:

1. If the player reached a game over, their current score would roll over to the next game, regardless of if they pressed new game or continue.
2. If the player reached a game over, there was a small chance that they could either not break anything regardless of mach, not be able to move at all, or both.
3. One of the testers had a wide-screen monitor, so when running the game, all the UI would be stretched out, meaning some elements such as buttons were off-screen or stretched out. (screenshots below)
4. When the player runs against a wall, any small-office doors could cause the player to knock over.
5. Players can enter the elevator and move around the edges of the elevator walls without causing the run to end.
6. Pressing spacebar and releasing sprint at the same time can give the player a large boost of speed. Some testers were able to do this consistently, while others couldn't, though most testers were able to do this at least once.

6.3. Post-testing changes

The first step in improving the project for the second wave of testing was bug fixing. Before this test, the collision detection for sprinting used a Spherecast, resulting in many collisions that felt too punishing, so I changed it to a Raycast [29]. I fixed multiple bugs regarding the Save File and UI display issues including the sizing of the UI layouts.

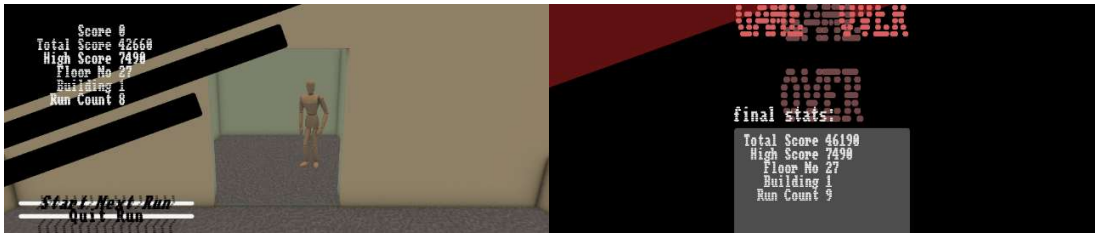


Figure 62, 63: Screenshots of the UI on a widescreen monitor before fixing.

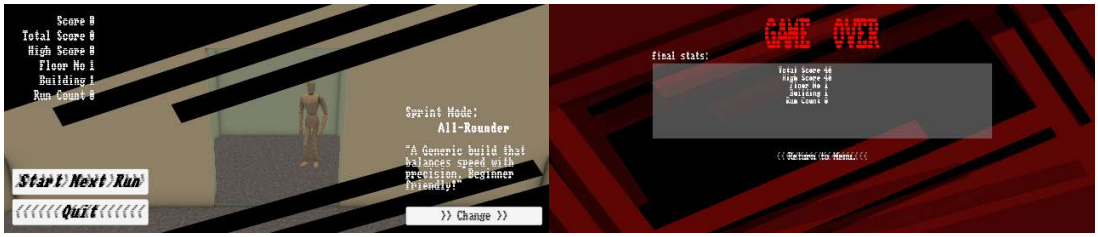


Figure 64, 65: Screenshots of the UI on a widescreen monitor after fixing.

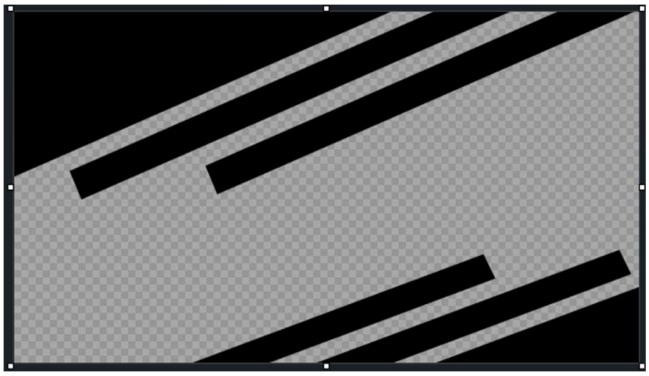


Figure 66: The new "Shutter" Texture used for the UI.

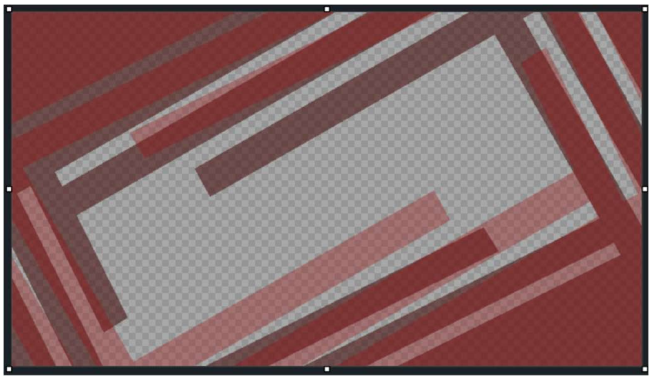


Figure 67: The new "Game Over Shutter" used for the Game Over UI.

I also overhauled the movement system, adding the option to press the space bar to stop boosts, or boost out of a stumble. This is because many testers were saying that stumbling is far too punishing or annoying. I also added the movement style customization ready for the second wave of testing.

6.4. User Testing and feedback – second wave

The second wave of testing was far smaller, only revisiting with two testers.

The first tester (previously tester 5) was now able to fully interact with the UI, as all the buttons were onscreen and sized correctly. The second tester (previously tester 1) was very happy with the movement customization. Both testers strongly enjoyed the dashing out of a stumble, saying that it felt "less punishing" and "more responsive".

During testing, bugs regarding the collision system were found and fixed. All the bug fixes as part of the first wave of testing seemed to have been successful.

7. Evaluation

7.1. What went well

Overall, Going Up has worked as fantastic proof of concept for a larger-scale game of this style. Everyone so far has really enjoyed the movement system, and the two testers who interacted with the movement style system really enjoyed it. The procedural generation system has also fit well into the mix – hardly any bugs erose from it, and it was always consistent.

The visual style is also effective. The simple colors and camera angle make the experience easy to get into. The animations of the player are very effective and make the sprint very fun to use. The subtle color differences of the props on each floor, and the expansiveness of each floor make a very large, almost endless atmosphere.

7.2: Future Work

7.2.1 Improvements

One of the main improvements I would make to the current system is to make each round longer, maps smaller, and props give less points. This is because many of the testers only reached one large office and returned. There was never any incentive to move deeper into each floor, mainly because there simply wasn't enough time to do so.

Beyond this, I would add particles to the player while running or props when broken. This would add far more responsiveness to these actions, increasing the fun value of the full game. I would also give each sprint style far more character, giving each custom animation and differentiating them all from each other more. I didn't to this originally due to time constraints, but giving this feature far more love would be a massive benefit for the replayability and gameplay customization of the final product.

7.2.2 Going forward

Given more time, the first thing I would add is an upgrade system. Finding special, rare props in the map to gain new stat boosts such as higher sprint speed or acceleration. This could also add one-time abilities such as a grapple. This would make the game far deeper, as players could plan what to take to each floor.

Finally, adding enemies that interact with the movement system directly. For example, a large fan that would blow the player away, allowing the player to either use it as a boost away from it, or struggle to go past it. The player stumbles enough just hitting walls, so the enemies should not be as punishing to the player unless they are entirely avoidable.

8. Conclusion

Overall, I am very happy with the prototype that has come out of this project. The biggest challenge with this project was making the systems work well together. The procedural generation and movement system are very complex systems and though they work well together, they don't exactly complement each other out of the box, leading to a lot of work on adjusting each of them to help them fit together. This project – in my mind at least – has been a success. It has been proven that these systems can work both individually and together, and I'm planning on taking what I've learnt here forward into more projects.

9. Appendices

9.1. References

1- Roguelikes & Roguelite: Szabados, G. Et al, 2022. "Roguelike games: The way we play."

2- Pizza Tower (Steam page): https://store.steampowered.com/app/2231450/Pizza_Tower/- Last accessed (21/04/2026)

3- Lethal Company (Steam page): https://store.steampowered.com/app/1966720/Lethal_Company/- Last accessed (21/04/2026)

4- Rogue (Steam page): https://store.steampowered.com/app/1443430/Rogue/ - Last accessed (21/04/2026)

5- R. van der Linden, "Procedural Generation of Dungeons," in IEEE Transactions on Computational Intelligence and AI in Games, March 2014

6- Tirkkonen, E., 2024. Randomness in roguelike games.

7- Ryan Watkins, "procedural content generation for unity game development" 2016, online via: https://books.google.co.uk/books?=en&lr=&id=8GwdDAAAQBAJ&oi=fnd&pg=PP1&dq - Last accessed (21/04/2026)

8- BCS code of conduct - https://www.bcs.org/membership-and-registrations/become-a-member/bcs-code-of-conduct/ - Last accessed (21/04/2026)

9- Wario Land 1 - https://www.nintendo.com/en-gb/Games/Game-Boy/Wario-Land - Last accessed (21/04/2026)

10- Spriters (Peppino) - https://www.spriters-resource.com/pc_computer/pizzatower/asset/133492/ - Last accessed (21/04/2026)

11- Mirrors Edge (Steam page) - https://store.steampowered.com/app/17410/Mirrors_Edge/ - Last accessed (21/04/2026)

12- Meinel, "Playing the Urban Future", 2019, online via: https://www.degruyterbrill.com/document/doi/10.1515/9783110659405-006/html - Last accessed (21/04/2026)

13- muma-87 challenge - https://www.reddit.com/r/lethalcompany/comments/1adx6k8/week_4_challenge_moon_muma87_layout/ - Last accessed (21/04/2026)

14- quake (Steam page) - https://store.steampowered.com/app/2310/Quake/ - Last accessed (21/04/2026)

15- id software - https://www.idsoftware.com/en - Last accessed (21/04/2026)

16- Mia Ivy - "Quake's Player Movement" 2023, Online via: https://github.com/myria666/qMovementDoc/blob/main/Quake_s_Player_Movement.pdf - Last accessed (21/04/2026)

17- source engine community documentation - https://developer.valvesoftware.com/wiki/SDK_Docs - Last accessed (21/04/2026)

18- Murray Spiegel "Vector Analysis" 1959, online via: http://debracollege.dspaces.org/bitstream/123456789/410/1/Vector%20Analysis%2C%20Spigel.pdf - Last accessed (21/04/2026)

19- Andrei Neacsu - "Recreating Quake/GoldSrc Movement in Godot 4.0" 2023, Online via: https://aneacsu.com/blog/2023-04-09-quake-movement-godot - Last accessed (21/04/2026)

20- Buttfield-Addison, P., Manning, J., & Nugent, T. (2023). Unity development cookbook : real-time solutions from game development to AI (Second edition.). O'Reilly Media, Incorporated.

21- unity - https://unity.com/ - Last accessed (21/04/2026)

22- github - https://github.com /- Last accessed (21/04/2026)

23- blender - https://www.blender.org/ - Last accessed (21/04/2026)

24- ProBuilder Manual - https://docs.unity3d.com/Packages/com.unity.probuilder@6.0/manual/index.html - Last accessed (21/04/2026)

25- Photopea - https://www.photopea.com/ - Last accessed (21/04/2026)

26- audacity - https://www.audacityteam.org/ - Last accessed (21/04/2026)

27- golemm - https://soundcloud.com/golemm - Last accessed (21/04/2026)

28 – Scriptable Object Unity Documentation - https://docs.unity3d.com/6000.4/Documentation/Manual/class-ScriptableObject.html - Last accessed (26/04/26)

29 – Raycast Unity Documentation - https://docs.unity3d.com/6000.4/Documentation/ScriptReference/Physics.Raycast.html - Last accessed (26/04/26)

9.2. Asset Credits

9.2.1. Models

Office props: https://sketchfab.com/3d-models/60s-office-props

- Have been edited (less polygons, new materials).

Player character model: https://sketchfab.com/3d-models/mannequin-anatomy-aid

- Has been edited (new animations, different texture).

9.2.2. Music and sounds

Music - Golemm: YouTube, SoundCloud

- No edits were made.
- Songs used: Untouchable, I Am, Dance Floor, Two Morningstarz

Object break and other sound effects:

Jug Break

Metal break

Bump on wall

Wood break 1

Wood break 2

- All edited down into individual clips

9.3. Project Proposal

3D Platformer Roguelike:

"Going Up?" - A 3D Platformer Roguelike with complex level generation and player controls to prioritize skill in the form of risk vs reward.*

Student Name:	Callum Hemsley
Student Email:	ch742@sussex.ac.uk
Program:	Computer Science: Multimedia and Games BSc
Candidate Number:	278905
Supervisor:	Charlotte Robinson

This Project has risen from my own personal appreciation of modern roguelite games and platformers that all take very generic genres and make a very interesting spin on these concepts. Great examples of these are:
Pizza Tower: a 2D platformer with very complex but unique movement mechanics that I aim to recreate in 3D
Roguelikes: Games such as Lethal Company or R.E.P.O that require the player to reach ever increasing goals to continue progression through procedurally generated levels with unique styles.

Aims

The primary objective is the mainly re-frame the pizza-tower movement into a flat, 3D environment in an intuitive way, and create a fun rogue-like gameplay loop to keep players engaged.
Additionally, all visuals (such as environments, characters, items, GUI and HUD) and audio will thematically fit and elevate the gameplay experience.
To do this, existing systems will be used to assist development. Examples would be the infamous Quake III movement to ensure more time can be spent on the more personalized features.

Objectives

Primary Objectives:

1. Create an engaging and re-playable gameplay loop using complex movement mechanics and level design while also ensuing the player is not overwhelmed by new mechanics.
2. Create an easy-to-use GUI and HUD to keep the player engaged using simplistic design and professional design with sharp feedback as to not either alienate or confuse the users.

3. Prioritize input-feedback to ensure that the player always feels their inputs have a result, and that the game reacts accordingly as expected. Movement should always feel snappy, UI should be responsive and readable, and in game triggers should be instantaneous and noticeable, while not unexpected.
4. Ensure that these objectives are met satisfactorily to end users via user-testing and questionnaires, while allowing users to play without supervision over long periods of time to ensure that results are ensured over long periods of gameplay.

Extensions:

1. Widen the game for more features such as more enemies or new personal upgrade options to ensure the gameplay loop does not become stale overtime, and users stay engaged.
2. Allow for more customization of the experience, such as character customisation, control bindings, and enhanced accessibility settings.
3. Add more variation to each individual level to keep the experience fluid, allowing for the player to be subtilty rewarded for exploration. This could be done via hidden secrets such as easter eggs, optional quests unique to certain areas, or simply use different assets/tiles for a floor to allow for visual distinction between floors or buildings.
4.

Relevance

This Project aims to mesh many of the main development styles throughout my course, including 3D graphics, multimedia, and 3D modelling for the main game itself. Other techniques such as agile development cycles for user-first design, organisation of user-testing, and requirement interviews will also be used to assure that all features are satisfactory for the end users.
As someone who also enjoys games such as this (Pizza Tower, “friend slop”, etc), alongside simply enjoying the concept this brings, the project will incorporate my own personal interests. This will provide me the support to leverage the games design myself without fully relying on user testing to ensure that the target audience will be satisfied.

Resources Required

This project is very software fronted, so access to machines that can access both Unity and other software such as Blender or Audacity for asset creation is needed. Alongside this, seminar and study room bookings for user-testing and interviews.

Weekly Timetable and Timescale

	Mon	Tues	Wed	Thurs	Fri	Sat + Sun
9	Lab		Lab		Lecture	
10	Lab		Lab			Minimum
11						6 hours
12		Minimum	Minimum	Lab		work

13		2 hours	2 hours	Lab		across
14		work	work			both days
15					Lecture	
16	Lecture					(3 per day)
17	Till 8					

Bibliography

Mia Ivy - “Quake’s Player Movement” 2023, Online via: https://github.com/myria666/qMovementDoc/blob/main/Quake_s_Player_Movement.pdf “ Quakes_s_Player_Movement.pdf”

Andrei Neacsu - “Recreating Quake/GoldSrc Movement in Godot 4.0” 2023, Online via: https://aneacsu.com/blog/2023-04-09-quake-movement-godot

R. van der Linden, R. Lopes and R. Bidarra, "Procedural Generation of Dungeons," in IEEE Transactions on Computational Intelligence and AI in Games, vol. 6, no. 1, pp. 78-89, March 2014, doi: 10.1109/TCIAIG.2013.2290371.

Joonas Anttila “Creating an adaptive camera system for a 3D platformer game” 2025, Online via: https://www.theseus.fi/bitstream/handle/10024/95047/Anttila_Joonas.pdf?sequence=1

Buttfield-Addison, P., Manning, J., & Nugent, T. (2023). *Unity development cookbook : real-time solutions from game development to AI* (Second edition.). O’Reilly Media, Incorporated.

Szabados, G., Bácsné Bába, É., Fenyves, V., Bács, Z., Molnár, A., Ráthonyi, G.G., Hussain, M.R. and Orbán, S.G., 2022. Roguelike games: The way we play.

Tirkkonen, E., 2024. Randomness in roguelike games.

9.4. User Testing Compliance Form

Software Testing With Users Compliance Form – UG & PGT Projects[1]

School of Engineering and Informatics, University of Sussex

This form should be used in conjunction with the document entitled "Ethical Review Guidance for Student Projects".

Prior to conducting your project, you and your supervisor must discuss the ethical implications of your project. If your proposed project complies with **all** of the points in this form, then may use this form.

If you use this form, you must also prepare either (a) an information sheet and a consent for your participant to sign, or (b) prepare an Introduction and Debriding Script to read to your participant before the test (see appendix below). These must be approved by you supervisor.

Include the signed copy of this form with your final project report/dissertation.

If your proposed project does not comply with all the points, you should refer back to the "Research Ethics Guidance for UG and PGT Projects" document for further guidance.

---

1. Participants were not exposed to any risks greater than those encountered in their normal working life.

You have a responsibility to protect participants from physical, mental and emotional harm during the testing. The risk of harm must be no greater than in ordinary life. Areas of potential risk that require ethical approval include, but are not limited to, investigations that require participant mobility (e.g. walking, running, use of public transport), unusual or repetitive activity or movement, physical hazards or discomfort, emotional distress, use of sensory deprivation (e.g. ear plugs or blindfolds), sensitive topics (e.g. sexual activity, drug use, political behaviour, ethnicity) or those which might induce discomfort, stress or anxiety (e.g. violent video games), bright or flashing lights, loud or disorienting noises, smell, taste, vibration, or force feedback.

2. The study materials are paper-based or comprises software running on standard hardware.

Participants should not be exposed to any risks associated with the use of non-standard equipment: anything other than pen-and-paper, standard PCs, mobile phones, and tablet computers is considered non-standard.

3. All participants must explicitly state that they agreed to take part, and that their data can be used in the project.

Participants cannot take part in the test without their knowledge or consent (i.e. no covert observation). Covert observation, deception or withholding information are deemed to be high risk and require ethical approval through the relevant F-REC.

If the results of the evaluation are likely to be used beyond the term of the project (for example, the software is to be deployed, the data is to be published or there are future secondary uses of the data), then it will be necessary to obtain signed consent from each participant. Otherwise, verbal consent is sufficient, and should be explicitly requested in the introductory script (see Appendix 1).

4. No incentives will be offered to the participants.

The payment of participants must not be used to induce them to risk harm beyond that which they risk without payment in their normal lifestyle. People volunteering to participate may be compensated financially e.g. for reasonable travel expenses. Payments made to individuals must not be so large as to induce individuals to risk harm beyond that which they would usually undertake.

5. No information about the evaluation or materials is intentionally withheld from the participants.

Withholding information from participants or misleading them is unacceptable without justifiable reasons for doing so. Any projects requiring deception (for example, only telling participants of the true purpose of the study afterwards so as not to influence their behaviour) are deemed high risk and require approval from the relevant F-REC.

6. No participant is under the age of 18.

Any studies involving children or young people are deemed to be high risk and require ethical approval through the relevant F-REC.

7. No participant has a disability or impairment that may have limited their understanding or communication or capacity to consent.

Projects involving participants with disabilities are deemed to be high risk and require ethical approval from the relevant F-REC.

8. Neither I nor my supervisor are in a position of authority or influence over any of the participants.

A position of authority or influence over any participant must not be allowed to pressurise participants to take part in, or remain in, any study.

9. All participants will be informed that they can withdraw at any time.

All participants have the right to withdraw at any time during the investigation. They should be told this in the introductory script (see Appendix 1).

10. All participants will be informed of my contact details, and the contact details of my supervisor.

All participants must be able to contact the investigator and/or the supervisor after the investigation. They should be given contact details for both student and supervisor as part of the debriefing.

11. The evaluation will be described in detail with all of the participants at the beginning of the session, and participants will be fully debriefed at the end of the session. All participants will be given the opportunity to ask questions at both the beginning and end of the session.

Participants must be provided with sufficient information prior to starting the session, and in the debriefing, to enable them to understand the nature of the investigation.

12. All the data collected from the participants will be stored securely, and in an anonymous form.

All participant data (hard-copy and soft-copy) should be stored securely (i.e. locked filing cabinets for hard copy, password protected computer for electronic data), and in an anonymised form.

Project title: *Going Up*

Student's Name: *Callum Hemsley*

Student's Registration Number: *22301603*

Student's Signature: _____

Date: *24/3/2026*

Supervisor's Name: ___*Charlotte Robinson*_____

I have read and authorised either (a) the participant information sheet and consent form or (b) the Introduction and Debrief scripts. **Supervisor's Initials:** ___*CR*___

Supervisor's Signature: _____

Date: ____*24/3/2026*_____

Software Testing With Users Compliance Form

Appendix 1: Introduction and Debriefing Scripts

If you intend to obtain verbal consent from your participants, you need to create an introduction script, to read out at the start of the study, and a debriefing script, to read out at the end of the study. You should get your supervisor's approval for your introduction and debriefing scripts before commencing your study.

NB: When submitting the
Signed Software Testing With Users Compliance form include your Introduction and Debriefing script.

Introduction Script

The introduction script must:

- state the general aim of the study
- explain why you need the involvement of other people
- describe what will happen in the study
- describe what data will be collected
- reassure the participant that any data collected will be stored securely, and in an anonymous format
- explain what interaction the participant may have with you during the study
- reassure the participant that this is not a test of ability
- state that the participant may withdraw at any time
- seek explicit consent
- allow the participant to ask questions

An example introductory script (italics indicate required information, some of which will be specific to your project):

Web Interface Investigation

Final Year Project, 2020-21

Jane Student

The aim of this study is to investigate the [*state the general aim of the experiment – e.g., suitability of a new website*]. We cannot tell how good this [. . .] is unless we ask those people who are likely to be using it, which is why we need to run studies like these [*explain why you need the involvement of other people*]. I will give you some time to [*describe the tasks in involved in general terms*], before asking you to answer some questions [*describe kinds of questions*].

I will be observing you while you perform the tasks [*describe what data will be collected*]. If you have any questions, please ask me, and please let me know when you are finished [*explain what interaction the participant may have with you during the study*].

I will ask you some questions at the end of the study [*describe what will happen in the study*]. Please remember that it is the system, not you, that is being evaluated [*reassure the participant that this is not a test of ability*].

You are welcome to withdraw from the study at any time [*state that the participant may withdraw at any time*]. Please be assured that any data collected will be stored securely and in an anonymous form [*describe how the data will be anonymised and stored*].

Do you agree to take part in this evaluation? [*seek explicit consent*]. Do you have any questions before we start? [*allow the participant to ask questions*].

Debriefing Script

The debriefing script that is used at the end of the study must:

- restate the main aim of the experiment
- if applicable, explain any other, related aims of the experiment, and any particular data collected
- allow the participant to make comments or ask questions
- give contact details for the student and supervisor
- thank the participant

An example debriefing script (italics indicate required information, some of which will be specific to your project):

Web Interface Investigation

Final Year Project, 2020-21

Jane Student

The main aim of the experiment was to investigate the suitability of this website [*restate the main aim of the experiment*]. In particular, I was looking to see whether you made use of the site map, and whether any of the links on the website seemed to be confusing. I wanted to know how easy the pages were to navigate [*if applicable, explain any other, related aims of the experiment, and any particular data collected*].

Do you have any comments or questions about the experiment? [*allow the*

participant to make comments or ask questions]. Here are my contact details, and those of my supervisor, and please let us know if you have any further questions about this study [*give contact details of student and supervisor to participant*]. Thank you for your help [*thank the participant*].

9.5. Supervisor Communications Calendar

01/10/2025 – Project accepted

17/10/2025 – Project proposal completed

21/10/2025 – Initial meeting with supervisor

3/11/2025 – communication with supervisor regarding ethical compliance

13/11/2025 – Interim report completed

14/3/2026 – meeting with supervisor regarding testing wave 1 and 2

15/4/2026 – meeting with supervisor regarding final report

9.6. Project Links

Project GitHub: https://github.com/Demented-101/Going-Up-Proj